ACM / OI 算法板子合集

C++17 常用算法模板 · 由 gen-templates-html.js 自动生成

生成日期：2026/9/5 | 模板数：45 | 源：**板子/** 目录

目录

基础

- 快读快写

数学

- 矩阵快速幂
- 快速幂
- 线性筛素数
- 组合数与逆元
- exgcd
- gcd与lcm

数据结构

- 并查集
- 单调队列
- 单调栈
- 树状数组
- 线段树
- 字典树
- ST表

图论

- 存图
- 二分图匹配_匈牙利
- 拓扑排序
- 网络流_Dinic
- Dijkstra
- Floyd
- Kruskal
- LCA
- SPFA
- Tarjan强连通分量

字符串

- 字符串哈希
- KMP
- Manacher

搜索

0-1BFS
记忆化搜索
BFS
DFS

DP

0-1背包
多重背包
区间DP
树形DP
数位DP
完全背包
状压DP
LCS
LIS

杂项

二分
二维前缀和
前缀和与差分
三分
双指针

基础

基础

快读快写

快读快写 — 基于 fread 缓冲的快速输入输出，应对大规模数据 IO

```cpp
// 快读快写 – 基于 fread 缓冲的快速输入输出，应对大规模数据 IO

#include <bits/stdc++.h>
using namespace std;

/*
 * 快速输入输出类（fread 缓冲版）。
 * 适用于输入输出量极大（如 n、m 达 1e6 甚至更多）的题目，
 * 相比 cin/cout 能显著减少 IO 时间。
 *
 * 主要接口：
 *   bool readInt(int &x)         读入一个 int，成功返回 true
 *   bool readLong(long long &x)  读入一个 long long，成功返回 true
 *   bool readStr(string &s)      读入一个连续字符串（遇到空白分割），成功返回 true
 *   void writeInt(int x)         输出一个 int
 *   void writeLong(long long x)  输出一个 long long
 *   void writeStr(const string &s) 输出一个字符串
 *   void flush()                 将缓冲区内容一次性写入 stdout
 *
 * 原理：一次性从 stdin 读入一大块到 buffer，
 *       由 pos 指针逐个字符解析，避免频繁调用读函数。
 */
struct FastIO {
    static const int BUFSIZE = 1 << 20; // 缓冲区 1MB

    char buf[BUFSIZE];
    int pos = 0;   // buffer 中当前读取位置

    int len = 0;   // buffer 中有效字符数


    // 从 stdin 读入下一批字符到 buffer

    inline bool readNext() {
        return (pos < len) || (len = fread(buf, 1, BUFSIZE, stdin), pos = 0, pos < len);
    }

    // 读入一个整数（支持负数），成功返回 true，到达结尾返回 false

    bool readInt(int &x) {
        return readSigned(x);
    }

    // 读入一个 long long（支持负数），成功返回 true

    bool readLong(long long &x) {
        return readSigned(x);
    }

    // 通用有符号整数读取模板

    template <typename T>
    bool readSigned(T &x) {
        x = 0;
        char c;
        bool neg = false;
        // 跳过空白字符

        do {
            if (!readNext()) return false; // 已到文件尾

            c = buf[pos++];
```

```
    } while (c <= ' ');
    // 处理负号

    if (c == '-') {
        neg = true;
        if (!readNext()) return false;
        c = buf[pos++];
    }
    // 逐位累积数字

    for (; c >= '0' && c <= '9'; c = buf[pos++]) {
        x = x * 10 + (c - '0');
        if (!readNext()) break;
    }
    if (neg) x = -x;
    return true;
}

// 读入一个字符串（以空白字符为分隔），成功返回 true

bool readStr(string &s) {
    s.clear();
    char c;
    // 跳过前导空白

    do {
        if (!readNext()) return false;
        c = buf[pos++];
    } while (c <= ' ');
    // 累积直到遇到空白

    do {
        s.push_back(c);
        if (!readNext()) break;
        c = buf[pos++];
    } while (c > ' ');
    return true;
}

// 输出缓冲区

char out[BUFSIZE];
int opos = 0;

// 将一个字符放入输出缓冲（必要时先 flush）

inline void putChar(char c) {
    if (opos >= BUFSIZE - 1) flush();
    out[opos++] = c;
}

// 通用有符号整数输出

template <typename T>
void writeSigned(T x) {
    if (x == 0) { putChar('0'); return; }
    if (x < 0) { putChar('-'); x = -x; }
    char tmp[32]; int t = 0;
    while (x > 0) { tmp[t++] = char('0' + x % 10); x /= 10; }
    while (t--) putChar(tmp[t]);
}

void writeInt(int x)      { writeSigned(x); }
void writeLong(long long x) { writeSigned(x); }

// 输出字符串

void writeStr(const string &s) {
```

```
        for (char c : s) putChar(c);
    }

    // 将输出缓冲全部写出

    void flush() {
        if (opos > 0) {
            fwrite(out, 1, opos, stdout);
            opos = 0;
        }
    }

    // 析构时自动刷新

    ~FastIO() { flush(); }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 说明：FastIO 内含两块 1MB 缓冲（共约 2MB），

    //       放入静态区（static）而非栈，避免在大栈限制下爆栈。

    static FastIO io;

    // 演示：依次读入 a、b、c（整数）和一个字符串 s，然后原样/反向输出

    int a, b;
    long long c;
    string s;
    io.readInt(a);
    io.readInt(b);
    io.readLong(c);
    io.readStr(s);

    io.writeInt(a + b);
    io.putChar('\n');
    io.writeLong(c);
    io.putChar('\n');
    // 反向输出字符串，演示 writeStr 的用法

    reverse(s.begin(), s.end());
    io.writeStr(s);
    io.putChar('\n');

    io.flush();
    return 0;
}
```

数学

数学

矩阵快速幂

矩阵快速幂 — 矩阵乘法 + 矩阵快速幂（带取模），用于计算斐波那契

```cpp
// 矩阵快速幂 — 矩阵乘法 + 矩阵快速幂（带取模），用于计算斐波那契

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const ll MOD = 1000000007; // 常见大素数模

const int SZ = 2;          // 矩阵维度（本题用 2 阶）；可改为任意 N 阶


// 矩阵结构体，存 n*n 方阵（这里固定 2x2，用小写 n 表示阶数）

struct Matrix {
    ll a[SZ][SZ];
    int n; // 阶数


    // 构造：n 阶矩阵，默认清零；若传 identity 则置为单位阵

    Matrix(int _n = 2, bool identity = false) {
        n = _n;
        memset(a, 0, sizeof(a));
        if (identity) for (int i = 0; i < n; ++i) a[i][i] = 1;
    }
};

/*
 * 矩阵乘法：C = A * B (mod MOD)
 * 经典 O(n^3) 三重循环。n 为阶数。
 */
Matrix matMul(const Matrix &A, const Matrix &B) {
    Matrix C(A.n);
    for (int i = 0; i < A.n; ++i) {
        for (int k = 0; k < A.n; ++k) {
            if (A.a[i][k] == 0) continue; // 稀疏优化，可略

            for (int j = 0; j < A.n; ++j) {
                C.a[i][j] = (C.a[i][j] + A.a[i][k] * B.a[k][j]) % MOD;
            }
        }
    }
    return C;
}

/*
 * 矩阵快速幂：计算 A^k (mod MOD)
 * 与整数快速幂同理：二进制拆分指数，结果矩阵初始为单位阵。
 * 复杂度：O(n^3 * log k)。
 */
Matrix matPow(Matrix A, ll k) {
    Matrix res(A.n, true); // 单位阵

    while (k > 0) {
        if (k & 1) res = matMul(res, A);
        A = matMul(A, A);
        k >>= 1;
    }
    return res;
}
```

```cpp
/*
 * 用矩阵快速幂求斐波那契数 F(k)
 * F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2)。
 * 递推式表现为矩阵：
 *   [F(n+1)]   = [1 1]^n   [F(1)]
 *   [F(n)  ]     [1 0]     [F(0)]
 * 取 M = [[1,1],[1,0]]，则 M^n 的第一行第一列即为 F(n+1)，
 * 第一行第二列（或第二行第一列）即为 F(n)。
 * 要求 n>=0。
 */
ll fib(ll n) {
    if (n == 0) return 0;
    Matrix M(2);
    M.a[0][0] = 1; M.a[0][1] = 1;
    M.a[1][0] = 1; M.a[1][1] = 0;
    Matrix Mn = matPow(M, n);
    return Mn.a[0][1] % MOD; // = F(n)

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：输出 F(10)（模 1e9+7），期望 55

    ll k = 10;
    cout << "F(" << k << ") mod 1e9+7 = " << fib(k) << "\n";

    // 额外验证几个

    ll test[] = {0, 1, 2, 5, 20};
    for (ll x : test) {
        cout << "F(" << x << ") = " << fib(x) << "\n";
    }
    return 0;
}
```

数学

快速幂

快速幂 — 二进制快速幂（含取模）与费马小定理求逆元

```cpp
// 快速幂 — 二进制快速幂（含取模）与费马小定理求逆元

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

/*
 * 二进制快速幂：计算 a^b mod p
 * 原理：把指数 b 按二进制拆解，base 逐次自乘（base, base^2, base^4, ...），
 *       每当 b 对应位为 1 就把当前 base 乘进答案。
 * 时间复杂度：O(log b)。
 */
ll qpow(ll a, ll b, ll p) {
    ll res = 1 % p;
    a %= p;
    while (b > 0) {
        if (b & 1) res = res * a % p; // 当前位为 1，乘进答案

        a = a * a % p;                // base 平方

        b >>= 1;                      // 指数右移一位

    }
    return res;
}

/*
 * 费马小定理求逆元
 * 当 p 为大素数且 gcd(a,p)=1 时：a^(p-1) ≡ 1 (mod p)，
 * 所以 a 的逆元为 a^(p-2) mod p。
 * 时间复杂度：O(log p)。
 */
ll inv(ll a, ll p) {
    return qpow(a, p - 2, p);
}

const ll MOD = 1000;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：2^10 mod 1000

    ll base = 2, exp = 10;
    cout << "2^10 mod 1000 = " << qpow(base, exp, MOD) << "\n"; // 1024 % 1000 = 24


    // 演示费马小定理求逆元：求 3 在模 1e9+7 下的逆元

    const ll PM = 1000000007LL;
    ll ainv = inv(3, PM);
    cout << "3 * inv(3) mod 1e9+7 = " << (3LL * ainv % PM) << "\n"; // 期望 1

    return 0;
}
```

数学

线性筛素数

线性筛素数 — 欧拉（线性）筛，求 [1,N] 内所有素数

```cpp
// 线性筛素数 — 欧拉（线性）筛，求 [1,N] 内所有素数

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 10000000; // 筛的上限（按需调整，数组用全局避免栈爆）


int primes[MAXN]; // 素数表，下标从 1 开始（或从 0 记录个数）

int primeCnt = 0; // 素数个数

bool isComp[MAXN + 1]; // isComp[i]=true 表示 i 是合数（非素数），否则为素数


/*
 * 线性筛（欧拉筛）：求 [1,N] 内所有素数
 * 原理：从小到大枚举 i，若 i 未被标记为合数则为素数，加入表。
 *       然后用最小质因子去标记合数：枚举已有素数 p，标记 i*p，
 *       一旦 p 是 i 的最小质因子（i % p == 0）就停止。
 * 这样每个合数都会被其【最小质因子】唯一标记一次，故总复杂度 O(N)（线性）。
 *
 * 关键：isComp[i*p]=true 时，若 i%p==0 则说明 p 已经是 i 的最小质因子，
 *       更大的素数对应的 i*p 已由更小的质因子覆盖，不必再标记。
 */
void linear_sieve(int N) {
    isComp[1] = true;          // 1 不是素数

    for (int i = 2; i <= N; ++i) {
        if (!isComp[i]) primes[++primeCnt] = i; // i 是素数

        for (int j = 1; j <= primeCnt && i * primes[j] <= N; ++j) {
            isComp[i * primes[j]] = true;       // 标记 i*primes[j] 为合数

            if (i % primes[j] == 0) break;      // 最小质因子已到，停止

        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：输出 100 以内所有素数

    int N = 100;
    linear_sieve(N);
    cout << "100 以内的素数 (" << primeCnt << " 个):\n";
    for (int i = 1; i <= primeCnt; ++i) {
        cout << primes[i];
        if (i < primeCnt) cout << ' ';
    }
    cout << "\n";
    return 0;
}
```

数学 组合数与逆元

组合数与逆元 — 预处理阶乘与阶乘逆元，求 C(n,r) mod p（p 为大素数）

```cpp
// 组合数与逆元 — 预处理阶乘与阶乘逆元，求 C(n,r) mod p（p 为大素数）

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int MOD = 1000000007; // 大素数

const int MAXN = 1000000;   // n 的上限（约 1e6）


ll fact[MAXN + 1];  // fact[i] = i! mod p

ll ifact[MAXN + 1]; // ifact[i] = (i!)^{-1} mod p（i! 的逆元）


/*
 * 费马小定理快速幂：求 a^b mod p
 * 用于计算逆元：inv(a) = a^(p-2) mod p（p 为大素数）。
 */
ll qpow(ll a, ll b, ll p) {
    ll res = 1 % p;
    a %= p;
    while (b > 0) {
        if (b & 1) res = res * a % p;
        a = a * a % p;
        b >>= 1;
    }
    return res;
}

/*
 * 预处理阶乘与阶乘逆元
 * fact[0..N]，其中 fact[0]=1。
 * ifact[N]=fact[N] 的逆元，然后用 ifact[i-1]=ifact[i]*i 递推回去。
 * 复杂度：O(N + log p)。
 */
void precompute(int N) {
    fact[0] = 1;
    for (int i = 1; i <= N; ++i) fact[i] = fact[i - 1] * i % MOD;
    ifact[N] = qpow(fact[N], MOD - 2, MOD);       // 用费马小定理求 N! 的逆元

    for (int i = N; i >= 1; --i) ifact[i - 1] = ifact[i] * i % MOD;
}

/*
 * 组合数 C(n,r) mod p
 * C(n,r) = n! / (r! * (n-r)!) = fact[n] * ifact[r] * ifact[n-r] (mod p)
 * 根据费马小定理，除以 r! 即乘以其逆元。
 * 要求 0 <= r <= n <= N。
 */
ll C(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * ifact[r] % MOD * ifact[n - r] % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    precompute(MAXN);
```

```
    // 演示：输出 C(5,2) mod 1e9+7
    cout << "C(5,2) mod 1e9+7 = " << C(5, 2) << "\n"; // 10

    // 额外演示
    cout << "C(10,3) mod 1e9+7 = " << C(10, 3) << "\n"; // 120
    cout << "C(1000000,1) mod 1e9+7 = " << C(1000000, 1) << "\n"; // 1000000
    return 0;
}
```

数学

exgcd

exgcd — 扩展欧几里得算法，求 ax+by=gcd(a,b) 的一组整数解

```cpp
// exgcd — 扩展欧几里得算法，求 ax+by=gcd(a,b) 的一组整数解

#include <bits/stdc++.h>
using namespace std;

/*
 * 扩展欧几里得 exgcd
 * 输入 a，b；输出 x，y 使得 ax + by = gcd(a,b)。
 * 返回值是最大公约数 g。
 *
 * 原理：由欧几里得 gcd(a,b)=gcd(b,a%b) 递归。
 * 设下一层已求出 bx' + (a%b)y' = g，
 * 而 a%b = a - (a/b)*b，代入可得：
 *   bx' + (a - (a/b)*b)y' = g
 *   整理得 a*y' + b*(x' - (a/b)*y') = g
 * 所以本层解为 x = y'，y = x' - (a/b)*y'。
 *
 * 时间复杂度：O(log min(a,b))。
 * 说明：求得的只是一组特解，通解为
 *   x + k*(b/g)，y - k*(a/g)（k 为任意整数）。
 */
long long exgcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1; y = 0;
        return a;
    }
    long long g = exgcd(b, a % b, y, x); // 交换并递归：下一层 x'=y, y'=x

    y -= (a / b) * x;                    // 调整本层 y

    return g;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：解 3x + 7y = 1

    long long a = 3, b = 7, x, y;
    long long g = exgcd(a, b, x, y);
    cout << "gcd(3,7) = " << g << "\n";
    cout << "x = " << x << ", y = " << y << "\n";
    cout << "验证：3*" << x << " + 7*" << y << " = "
         << (a * x + b * y) << "\n";  // 期望输出 1

    return 0;
}
```

数学 gcd与lcm

gcd与lcm — 欧几里得最大公约数与最小公倍数

```cpp
// gcd与lcm — 欧几里得最大公约数与最小公倍数

#include <bits/stdc++.h>
using namespace std;

/*
 * 最大公约数 gcd —— 欧几里得算法（辗转相除法）
 * 原理：gcd(a,b) = gcd(b, a % b)，不断递归直到余数为 0。
 * 时间复杂度：O(log min(a,b))。
 * 参数需为正（或至少不为 0）；对 a==0 返回 |b|。
 */
long long gcd(long long a, long long b) {
    return b == 0 ? a : gcd(b, a % b);
}

/*
 * 最小公倍数 lcm
 * 原理：lcm(a,b) = a / gcd(a,b) * b。
 * 先除后乘避免 a*b 溢出 long long（当 a、b 很大时）。
 */
long long lcm(long long a, long long b) {
    return a / gcd(a, b) * b;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：输出 gcd(12,18) 与 lcm(12,18)

    cout << "gcd(12,18) = " << gcd(12, 18) << "\n";   // 期望 6

    cout << "lcm(12,18) = " << lcm(12, 18) << "\n";   // 期望 36


    // 额外演示一组

    cout << "gcd(3,7) = " << gcd(3, 7) << "\n";       // 期望 1

    cout << "lcm(3,7) = " << lcm(3, 7) << "\n";       // 期望 21

    return 0;
}
```

数据结构

数据结构

并查集

并查集 — 路径压缩 + 按大小合并 的 DSU（Find / Unite / 判定连通）

```cpp
// 并查集 — 路径压缩 + 按大小合并 的 DSU（Find / Unite / 判定连通）

#include <bits/stdc++.h>
using namespace std;

// fa[x]：x 的父节点（根节点满足 fa[x]==x）

// sz[x]：x 作为根时子树的大小（仅在根上有意义），用于按大小合并

const int MAXN = 100005;
int fa[MAXN], sz[MAXN];

// 带路径压缩的查找：返回 x 所在集合的根，并把沿途节点直接挂在根上

// 均摊复杂度：O(alpha(n))，近似常数

int find(int x) {
    if (fa[x] == x) return x;             // 自己是根

    return fa[x] = find(fa[x]);           // 路径压缩：递归找根并直接接根

}

// 初始化：每个元素自成一个集合，大小为 1

void init(int n) {
    for (int i = 1; i <= n; ++i) {
        fa[i] = i;
        sz[i] = 1;
    }
}

// 合并两个元素所在的集合，返回是否发生了真正的合并

// 按大小（sz）合并：把小集合接到大集合上，避免树过深

bool unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return false;           // 已在同一集合，无需合并

    if (sz[rx] < sz[ry]) swap(rx, ry);    // 保证 rx 是较大的那棵

    fa[ry] = rx;                          // 把 ry 接到 rx 上

    sz[rx] += sz[ry];                     // 更新新根的大小

    return true;
}

// 判定两个元素是否连通（是否在同一集合）

bool same(int x, int y) {
    return find(x) == find(y);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：10 个元素，依次合并若干对，输出每个元素所在集合的根

    int n = 10;
```

```
    init(n);

    unite(1, 2);
    unite(2, 3);
    unite(4, 5);
    unite(6, 7);
    unite(7, 8);
    unite(8, 9);
    unite(3, 5);  // 把 {1,2,3} 与 {4,5} 连成 {1,2,3,4,5}


    cout << "每个元素的集合根:" << endl;
    for (int i = 1; i <= n; ++i)
        cout << i << " -> " << find(i) << (i == n ? "\n" : "\n");

    // 判定连通性演示

    cout << "same(1,4) = " << same(1, 4) << " (应为 1，同属集合)" << endl;
    cout << "same(4,9) = " << same(4, 9) << " (应为 0，不同集合)" << endl;
    cout << "same(6,10) = " << same(6, 10) << " (应为 0，不同集合)" << endl;

    return 0;
}
```

数据结构

单调队列

单调队列 — 求定长滑动窗口的最大值（窗口大小 k）

```cpp
// 单调队列 — 求定长滑动窗口的最大值（窗口大小 k）

#include <bits/stdc++.h>
using namespace std;

// 用双端队列 deque 维护一个单调递减的序列，存储的是元素下标

// 队列里下标对应的值从左到右单调递减，队头始终是当前窗口最大值

// 复杂度：每个元素最多进队/出队一次，总体 O(n)

const int MAXN = 100005;
int a[MAXN];          // 原数组，1-indexed

int ans[MAXN];        // ans[i] 存储以 i 为右端点的窗口最大值


// 求所有长度为 k 的滑动窗口的最大值，窗口右端点为 i（i 从 k 到 n）

// 结果写入 ans[i]

void slidingWindowMax(int n, int k) {
    deque<int> dq;
    for (int i = 1; i <= n; ++i) {
        // 1) 弹出队尾所有值 <= 当前元素的（它们不再可能成为最大值，因为更靠前且更小）

        while (!dq.empty() && a[dq.back()] <= a[i])
            dq.pop_back();
        // 2) 当前元素入队

        dq.push_back(i);
        // 3) 弹出队头中已经滑出窗口（下标 < i-k+1）的元素

        while (!dq.empty() && dq.front() < i - k + 1)
            dq.pop_front();
        // 4) 窗口形成（i >= k）时，队头即该窗口最大值

        if (i >= k)
            ans[i] = a[dq.front()];
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：窗口大小为 3 的滑动窗口最大值

    int n = 8, k = 3;
    int data[9] = {0, 1, 3, -1, -3, 5, 3, 6, 7};  // a[1]..a[8]

    for (int i = 1; i <= n; ++i) a[i] = data[i];

    slidingWindowMax(n, k);

    cout << "原数组 a[1..8]:" << endl;
    for (int i = 1; i <= n; ++i)
        cout << a[i] << " \n"[i == n];

    cout << "窗口大小 k=" << k << "，各窗口最大值（窗口的右端点对应值）:" << endl;
    for (int i = k; i <= n; ++i)
        cout << "窗口[" << i - k + 1 << "," << i << "] 最大值 = " << ans[i] << endl;
```

```
    return 0;
}
```

数据结构

单调栈

单调栈 — 求每个元素 左边/右边 第一个更大/更小元素

```cpp
// 单调栈 — 求每个元素 左边/右边 第一个更大/更小元素

#include <bits/stdc++.h>
using namespace std;

// 用栈维护一个单调递增/递减的序列，栈中存放的是元素下标

// 复杂度：每个元素最多入栈/出栈一次，总体 O(n)

const int MAXN = 100005;
int a[MAXN];    // 原数组，1-indexed

int n;

// 求每个元素右边第一个严格更大元素的（下标），不存在则为 0

// 单调递减栈：栈底到栈顶单调递减（存下标，值递减）

// 当当前元素大于栈顶时，栈顶的"下一个更大"就是当前元素

void nextGreaterElement(int res[]) {
    stack<int> st;
    for (int i = 1; i <= n; ++i) {
        while (!st.empty() && a[st.top()] < a[i]) {  // 栈顶被当前更大值弹出

            res[st.top()] = i;                       // 记录"下一个更大"下标

            st.pop();
        }
        st.push(i);
    }
    while (!st.empty()) {                            // 剩余元素右侧没有更大值

        res[st.top()] = 0;
        st.pop();
    }
}

// 求每个元素左边第一个更大元素（下标），不存在则为 0

// 从左往右扫，维护单调递减栈：栈顶就是左边最近的更大者

void prevGreaterElement(int res[]) {
    stack<int> st;
    for (int i = 1; i <= n; ++i) {
        while (!st.empty() && a[st.top()] < a[i]) st.pop();
        res[i] = st.empty() ? 0 : st.top();
        st.push(i);
    }
}

// 求每个元素右边第一个更小元素（下标），不存在则为 0

// 单调递增栈：栈顶被当前更小值弹出

void nextSmallerElement(int res[]) {
    stack<int> st;
    for (int i = 1; i <= n; ++i) {
        while (!st.empty() && a[st.top()] > a[i]) {
            res[st.top()] = i;
            st.pop();
        }
```

```
        st.push(i);
    }
    while (!st.empty()) { res[st.top()] = 0; st.pop(); }
}

// 求每个元素左边第一个更小元素（下标），不存在则为 0

void prevSmallerElement(int res[]) {
    stack<int> st;
    for (int i = 1; i <= n; ++i) {
        while (!st.empty() && a[st.top()] > a[i]) st.pop();
        res[i] = st.empty() ? 0 : st.top();
        st.push(i);
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：下一个更大元素

    n = 7;
    int data[8] = {0, 2, 1, 5, 3, 4, 6, 7};  // a[1]..a[7]

    for (int i = 1; i <= n; ++i) a[i] = data[i];

    int nge[MAXN];
    nextGreaterElement(nge);

    cout << "原数组 a[1..7]:" << endl;
    for (int i = 1; i <= n; ++i)
        cout << a[i] << " \n"[i == n];

    cout << "每个元素右边第一个更大元素(下标) : 值" << endl;
    for (int i = 1; i <= n; ++i) {
        cout << "i=" << i << " (" << a[i] << ") -> "
             << (nge[i] ? to_string(nge[i]) + " (" + to_string(a[nge[i]]) + ")" : "没有")
             << endl;
    }

    return 0;
}
```

数据结构

树状数组

树状数组（Fenwick / BIT）— 单点加、前缀和、区间和

```cpp
// 树状数组（Fenwick / BIT）— 单点加、前缀和、区间和

#include <bits/stdc++.h>
using namespace std;

// bit[i] 维护原数组的一个区间和，区间长度为 lowbit(i)

// lowbit(x) = x & -x，即 x 二进制里最低位的 1 对应的值

const int MAXN = 100005;
int bit[MAXN];
int n;  // 元素个数


// 单点加：把下标 pos 的值加上 delta，向上更新所有包含 pos 的树状节点

// 复杂度：O(log n)

void add(int pos, int delta) {
    for (; pos <= n; pos += pos & -pos)
        bit[pos] += delta;
}

// 前缀和：求 a[1..pos] 的和，向下沿 lowbit 累加

// 复杂度：O(log n)

int sumPrefix(int pos) {
    int res = 0;
    for (; pos > 0; pos -= pos & -pos)
        res += bit[pos];
    return res;
}

// 区间和：求 a[l..r] 的和 = 前缀和(r) - 前缀和(l-1)

// 复杂度：O(log n)

int rangeSum(int l, int r) {
    return sumPrefix(r) - sumPrefix(l - 1);
}

// 由数组 a（1-indexed）建树状数组：对每个位置做一次单点加

// 复杂度：O(n log n)

void build(const vector<int>& a) {
    for (int i = 1; i <= n; ++i)
        add(i, a[i]);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：由数组建树状数组，并查询前缀和与区间和

    vector<int> a = {0, 1, 3, 5, 7, 9, 11};  // a[0] 占位不用，有效下标 1..6

    n = (int)a.size() - 1;                   // n = 6

    build(a);
```

```
    cout << "原数组 a[1..6]:" << endl;
    for (int i = 1; i <= n; ++i)
        cout << a[i] << " \n"[i == n];

    cout << "前缀和 sumPrefix(4) = " << sumPrefix(4) << " (应为 1+3+5+7=16)" << endl;
    cout << "区间和 rangeSum(2,5) = " << rangeSum(2, 5) << " (应为 3+5+7+9=24)" << endl;

    // 单点加演示：a[3] += 10

    add(3, 10);
    cout << "add(3,10) 后 rangeSum(1,3) = " << rangeSum(1, 3) << " (应为 1+13+5=19)" << endl;

    return 0;
}
```

数据结构

线段树

线段树 — 单点修改 + 区间求和（另附懒标记版：区间加 + 区间求和）

```cpp
// 线段树 — 单点修改 + 区间求和（另附懒标记版：区间加 + 区间求和）

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;

// ============ 基础版：单点修改 + 区间求和 ============

// tree[x] 存储节点 x 所代表区间的和

int tree[MAXN * 4];  // 4 倍空间足够容纳所有节点

int arr[MAXN];       // 原数组，1-indexed


// 建树：递归区间 [l, r]，节点编号 p

// 复杂度：O(n)

void build(int p, int l, int r) {
    if (l == r) {                // 叶子节点

        tree[p] = arr[l];
        return;
    }
    int mid = (l + r) >> 1;
    build(p << 1, l, mid);       // 左子节点

    build(p << 1 | 1, mid + 1, r);// 右子节点

    tree[p] = tree[p << 1] + tree[p << 1 | 1];  // 左右子树之和

}

// 单点修改：把下标 pos 的值改为 val，从根向下更新路径

// 复杂度：O(log n)

void update(int p, int l, int r, int pos, int val) {
    if (l == r) {                // 找到目标叶

        tree[p] = val;
        return;
    }
    int mid = (l + r) >> 1;
    if (pos <= mid) update(p << 1, l, mid, pos, val);
    else            update(p << 1 | 1, mid + 1, r, pos, val);
    tree[p] = tree[p << 1] + tree[p << 1 | 1];  // 回溯更新

}

// 区间求和：求 [ql, qr] 的和，递归覆盖查询区间

// 复杂度：O(log n)

int query(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return tree[p];       // 当前区间被完全覆盖

    int mid = (l + r) >> 1;
    int res = 0;
    if (ql <= mid) res += query(p << 1, l, mid, ql, qr);    // 左区间有交集
```

```
    if (qr > mid)  res += query(p << 1 | 1, mid + 1, r, ql, qr);  // 右区间有交集

    return res;
}

// ============ 懒标记版：区间加 + 区间求和 ============

// lazy[x] 表示节点 x 的区间上还有多少"未下传"的加值

long long lazy[MAXN * 4];   // 懒标记（用 long long 防溢出）

long long seg[MAXN * 4];    // 区间和


// 下传懒标记：把 lazy[p] 分给左右子节点，并清空自身

void pushDown(int p, int l, int r) {
    if (lazy[p] == 0) return;
    int mid = (l + r) >> 1;
    // 左子节点长度 mid-l+1，右子节点长度 r-mid

    lazy[p << 1]     += lazy[p];
    seg[p << 1]      += lazy[p] * (mid - l + 1);
    lazy[p << 1 | 1] += lazy[p];
    seg[p << 1 | 1]  += lazy[p] * (r - mid);
    lazy[p] = 0;   // 标记已下传

}

// 建树（懒标记版）：初始化为 arr

void buildLazy(int p, int l, int r) {
    lazy[p] = 0;
    if (l == r) { seg[p] = arr[l]; return; }
    int mid = (l + r) >> 1;
    buildLazy(p << 1, l, mid);
    buildLazy(p << 1 | 1, mid + 1, r);
    seg[p] = seg[p << 1] + seg[p << 1 | 1];
}

// 区间加：给 [ql, qr] 每个数加上 val

// 复杂度：O(log n)

void rangeAdd(int p, int l, int r, int ql, int qr, long long val) {
    if (ql <= l && r <= qr) {                // 完全覆盖，打懒标记即可

        seg[p] += val * (r - l + 1);
        lazy[p] += val;
        return;
    }
    pushDown(p, l, r);                       // 有交集才先下传

    int mid = (l + r) >> 1;
    if (ql <= mid) rangeAdd(p << 1, l, mid, ql, qr, val);
    if (qr > mid)  rangeAdd(p << 1 | 1, mid + 1, r, ql, qr, val);
    seg[p] = seg[p << 1] + seg[p << 1 | 1];
}

// 区间求和（懒标记版）

// 复杂度：O(log n)

long long rangeSum(int p, int l, int r, int ql, int qr) {
    if (ql <= l && r <= qr) return seg[p];
    pushDown(p, l, r);
    int mid = (l + r) >> 1;
```

```cpp
    long long res = 0;
    if (ql <= mid) res += rangeSum(p << 1, l, mid, ql, qr);
    if (qr > mid)  res += rangeSum(p << 1 | 1, mid + 1, r, ql, qr);
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：建树、单点修改、区间求和

    int n = 6;
    int data[6] = {1, 3, 5, 7, 9, 11};
    for (int i = 1; i <= n; ++i) arr[i] = data[i - 1];

    build(1, 1, n);
    cout << "基础版区间和:" << endl;
    cout << "query(2,5) = " << query(1, 1, n, 2, 5) << " (应为 3+5+7+9=24)" << endl;

    update(1, 1, n, 3, 10);   // 把 a[3] 改为 10

    cout << "把 a[3] 改为 10 后 query(1,4) = " << query(1, 1, n, 1, 4)
         << " (应为 1+3+10+7=21)" << endl;

    // 懒标记版：区间加 + 区间求和

    cout << "\n懒标记版（区间加 + 区间求和）:" << endl;
    buildLazy(1, 1, n);
    rangeAdd(1, 1, n, 2, 4, 2);    // 给 a[2..4] 各 +2

    cout << "rangeAdd(2,4,+2) 后 rangeSum(1,5) = " << rangeSum(1, 1, n, 1, 5)
         << " (应为 (1+3+5+7+9)+(2*3)=31)" << endl;
    rangeAdd(1, 1, n, 1, 6, -1);   // 再给全部各 -1

    cout << "rangeAdd(1,6,-1) 后 rangeSum(1,6) = " << rangeSum(1, 1, n, 1, 6)
         << " (应为 36，即第一次后数组为 1,5,7,9,9,11 和 42，再整体 -1 得 42-6=36)" << endl;

    return 0;
}
```

数据结构

字典树

字典树（Trie）— 插入、查找、统计前缀数量、删除单词

```cpp
// 字典树（Trie）— 插入、查找、统计前缀数量、删除单词

#include <bits/stdc++.h>
using namespace std;

// 每个节点：next 指向 26 个字母子节点，less_count 统计"经过该节点"的单词数，

// end_count 统计"以该节点结尾"的单词数

// 节点总数 = 单词总长度 + 1，用 vector / 数组动态分配

struct TrieNode {
    int next[26];      // 子节点下标，0 表示不存在

    int less_count;    // 以该节点为前缀的单词个数

    int end_count;     // 以该节点结尾的单词个数

    TrieNode() {
        memset(next, 0, sizeof(next));
        less_count = end_count = 0;
    }
};

vector<TrieNode> trie(1);  // 0 号节点为根


// 插入一个单词：沿路径建立节点并累加计数

// 复杂度：O(len)，len 为单词长度

void insert(const string& s) {
    int u = 0;   // 当前节点，从根出发

    for (char c : s) {
        int d = c - 'a';
        if (trie[u].next[d] == 0) {          // 没有该子节点则新建

            trie[u].next[d] = (int)trie.size();
            trie.emplace_back();
        }
        u = trie[u].next[d];
        trie[u].less_count++;                // 经过该节点，前缀计数 +1

    }
    trie[u].end_count++;                     // 单词结尾计数 +1

}

// 判断是否存在某个单词（完全匹配）

// 复杂度：O(len)

bool contains(const string& s) {
    int u = 0;
    for (char c : s) {
        int d = c - 'a';
        if (trie[u].next[d] == 0) return false;   // 路径断了

        u = trie[u].next[d];
    }
    return trie[u].end_count > 0;                 // 存在以该节点结尾的单词
```

```
}

// 统计以字符串 s 为前缀的单词数量

// 复杂度：O(len)

int countPrefix(const string& s) {
    int u = 0;
    for (char c : s) {
        int d = c - 'a';
        if (trie[u].next[d] == 0) return 0;       // 前缀不存在

        u = trie[u].next[d];
    }
    return trie[u].less_count;
}

// 删除一个单词：若能找到，则沿路径减少 less_count，并减少结尾计数

// 若删掉的节点不再被任何单词使用，这里只做计数清理（保留结构），

// 为简洁不失正确性，此处不回收节点内存

// 注意：函数命名用 removeWord，避免与 <cstdio> 里的 ::remove(const char*)（删除文件）同名冲突

// 复杂度：O(len)

bool removeWord(const string& s) {
    if (!contains(s)) return false;               // 单词不存在则无须删除

    int u = 0;
    for (char c : s) {
        int d = c - 'a';
        u = trie[u].next[d];
        trie[u].less_count--;
    }
    trie[u].end_count--;
    return true;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：插入若干单词并查询

    vector<string> words = {"apple", "apply", "app", "application", "banana"};
    cout << "依次插入:" << endl;
    for (const auto& w : words) {
        insert(w);
        cout << "  " << w << endl;
    }

    cout << "contains(\"apple\")   = " << contains("apple") << " (应为 1)" << endl;
    cout << "contains(\"appl\")    = " << contains("appl") << " (应为 0，appl 不是完整单词)" << endl;
    cout << "contains(\"banana\")  = " << contains("banana") << " (应为 1)" << endl;

    cout << "countPrefix(\"app\")        = " << countPrefix("app") << " (应为 4: apple/apply/app/application)" <<
endl;
    cout << "countPrefix(\"appl\")       = " << countPrefix("appl") << " (应为 3: apple/apply/application)" <<
endl;
    cout << "countPrefix(\"ban\")        = " << countPrefix("ban") << " (应为 1: banana)" << endl;

    // 删除演示

    cout << "删除 \"app\" 后 countPrefix(\"app\") = " << (removeWord("app") ? countPrefix("app") : -1)
         << " (应为 3，删掉 app 后剩 apple/apply/application)" << endl;
```

```
    cout << "再次删除不存在的 \"xyz\" 返回 " << removeWord("xyz") << " (应为 0)" << endl;

    return 0;
}
```

数据结构

ST表

ST 表（稀疏表 / Sparse Table）— 静态区间 RMQ，查询区间最大值

```cpp
// ST 表（稀疏表 / Sparse Table）— 静态区间 RMQ，查询区间最大值

#include <bits/stdc++.h>
using namespace std;

// st[j][i]：从下标 i 开始、长度为 2^j 的区间的最大值

// 由于每行长度不同，这里用 vector 动态建表

// 复杂度：预处理 O(n log n)，单次查询 O(1)

const int MAXN = 100005;
int a[MAXN];                // 原数组，1-indexed

vector<int> st[20];         // log2(1e5) < 17，开 20 足够

int lg[MAXN];               // lg[i] = floor(log2(i))


// 预处理：先求每个数的 log2，再逐层递推 ST 表

void buildST(int n) {
    // 预处理 log2 表

    lg[1] = 0;
    for (int i = 2; i <= n; ++i)
        lg[i] = lg[i >> 1] + 1;

    // 第 0 层：长度为 1 的区间，最大值即原数组

    st[0].resize(n + 1);
    for (int i = 1; i <= n; ++i) st[0][i] = a[i];

    // 递推：st[j][i] = max(st[j-1][i], st[j-1][i + 2^(j-1)])

    for (int j = 1; (1 << j) <= n; ++j) {
        int len = 1 << j;               // 当前区间长度 2^j

        st[j].resize(n + 1);
        for (int i = 1; i + len - 1 <= n; ++i)
            st[j][i] = max(st[j - 1][i], st[j - 1][i + (len >> 1)]);
    }
}

// 查询区间 [l, r] 的最大值

// 取两个可重叠的长度为 2^k 的区间覆盖 [l,r]，k = floor(log2(r-l+1))

// 复杂度：O(1)

int queryMax(int l, int r) {
    int k = lg[r - l + 1];
    return max(st[k][l], st[k][r - (1 << k) + 1]);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：对数组建 ST 表，查询若干区间最大值

    int n = 10;
```

```
    int data[12] = {0, 5, 2, 9, 3, 7, 1, 8, 6, 4, 10};  // a[1]..a[10]

    for (int i = 1; i <= n; ++i) a[i] = data[i];

    buildST(n);

    cout << "原数组 a[1..10]:" << endl;
    for (int i = 1; i <= n; ++i)
        cout << a[i] << " \n"[i == n];

    cout << "queryMax(1,10) = " << queryMax(1, 10) << " (应为 10)" << endl;
    cout << "queryMax(2,6)  = " << queryMax(2, 6) << " (应为 9，区间 2,9,3,7,1)" << endl;
    cout << "queryMax(7,9)  = " << queryMax(7, 9) << " (应为 8，区间 8,6,4)" << endl;
    cout << "queryMax(4,4)  = " << queryMax(4, 4) << " (应为 3，单点)" << endl;

    return 0;
}
```

图论

图论

存图

存图 — 邻接表存图：vector<pair> 邻接表 与 链式前向星(head/ver/next) 两种风格

```cpp
// 存图 — 邻接表存图：vector<pair> 邻接表 与 链式前向星(head/ver/next) 两种风格

#include <bits/stdc++.h>
using namespace std;

/*
 * 两种常用存图方式对比：
 * （1）vector<pair<int,int>> 邻接表 — 写法简单、易遍历、易扩展（可加额外信息），
 *       适合边数较大且遍历次数多的场合；常数略大，但通常足够。
 * （2）链式前向星（head/ver/next） — 用静态数组模拟链表，常数小、内存连续，
 *       适合边数大、卡时间的场合；经典竞赛写法，需要手动加边函数。
 * 本文件演示两种风格，均读入同一张带权小图并输出边长。
 */

const int MAXN = 1005;
const int MAXM = 2005; // 无向图边数 = 2 * 输入边数，这里留足余量


// ---------- 风格一：vector<pair<int,int>> 邻接表 ----------

// table[u] 中每个元素为 (v, w)，表示 u--v 有一条权为 w 的边

struct Adj {
    vector<pair<int,int>> table[MAXN];
    void addEdge(int u, int v, int w) { table[u].push_back({v, w}); }
    // 遍历 u 的所有出边：for (auto [v,w] : table[u]) ...

};

// ---------- 风格二：链式前向星 ----------

struct ForwardStar {
    int head[MAXN];   // head[u] 指向 u 的第一条边的下标，-1 表示没有边

    int ver[MAXM];    // ver[i] 第 i 条边指向的点 v

    int nxt[MAXM];    // nxt[i] 第 i 条边的下一条边（同一起点的下一条）

    int w[MAXM];      // w[i] 第 i 条边的权

    int tot;          // 当前已加的边数（边下标从 0 开始）


    void init(int n) {
        tot = 0;
        for (int i = 1; i <= n; ++i) head[i] = -1; // 用 -1 表示空

    }
    // 加一条有向边 u->v，权为 w。若要无向，则再倒着加一条 v->u。

    void addEdge(int u, int v, int val) {
        ver[tot] = v;
        w[tot]   = val;
        nxt[tot] = head[u]; // 头插法，新边插到最前面

        head[u]  = tot++;
    }
    // 遍历 u 的所有出边：for (int e = head[u]; e != -1; e = nxt[e]) ...

};

int main() {
```

```cpp
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：读入一张带权小图并分别用两种存图方式输出每条边

    // 输入格式：

    //   第一行 n m（点数、边数，无向图）

    //   接下来 m 行：u v w

    int n, m;
    cin >> n >> m;

    Adj a;                  // 邻接表

    ForwardStar fs;         // 链式前向星

    fs.init(n);

    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        a.addEdge(u, v, w);  // 邻接表加一条（此处按无向演示可双向加）

        a.addEdge(v, u, w);
        fs.addEdge(u, v, w); // 链式前向星同理加双向

        fs.addEdge(v, u, w);
    }

    // 输出风格一：邻接表

    cout << "--- 邻接表 (vector<pair>) ---\n";
    for (int u = 1; u <= n; ++u) {
        cout << u << ":";
        for (auto [v, w] : a.table[u]) cout << " -> (" << v << ", " << w << ")";
        cout << "\n";
    }

    // 输出风格二：链式前向星

    cout << "--- 链式前向星 (head/ver/next) ---\n";
    for (int u = 1; u <= n; ++u) {
        cout << u << ":";
        for (int e = fs.head[u]; e != -1; e = fs.nxt[e])
            cout << " -> (" << fs.ver[e] << ", " << fs.w[e] << ")";
        cout << "\n";
    }
    return 0;
}
```

图论 二分图匹配_匈牙利

二分图最大匹配 — 匈牙利算法（Kuhn，基于增广路）

```cpp
// 二分图最大匹配 — 匈牙利算法（Kuhn，基于增广路）
#include <bits/stdc++.h>
using namespace std;

/*
 * 匈牙利算法（Kuhn）求二分图最大匹配：
 *   对左部点依次尝试找一条"增广路"（匹配边/非匹配边交替出现，
 *   且两端都是未匹配点的路径），找到则匹配数 +1。
 *
 *   vis 数组记录"本轮"访问过的右部点，每个左部点开始时清空，
 *   用于防止 DFS 死循环；只对右部点打标记。
 *
 *   复杂度 O(nE)，n 为左部点数。对一般模板题足够。
 *   也可用网络流建模求最大匹配（见 网络流_Dinic.cpp）。
 */

const int MAXN = 1005;

int n, m;             // 左部点 1..n，右部点 1..m
vector<int> G[MAXN];  // 左部点 -> 可匹配的右部点
int matchR[MAXN];     // 右部点当前匹配的左部点，0 表示未匹配
int vis[MAXN];        // 本轮右部点访问标记

// 给左部点 u 找增广路，找到返回 true
bool dfs(int u) {
    for (int v : G[u]) {
        if (vis[v]) continue;
        vis[v] = 1;
        // v 未匹配，或 v 当前匹配的点能换到别处
        if (matchR[v] == 0 || dfs(matchR[v])) {
            matchR[v] = u;
            return true;
        }
    }
    return false;
}

// 返回最大匹配数；匹配方案保存在 matchR 里
int hungarian() {
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
        memset(vis, 0, sizeof(vis));
        if (dfs(i)) ++ans;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // P3386 输入：n m k，接下来 k 行 u v（左部 u 连向右部 v）
    int k;
    cin >> n >> m >> k;
    for (int i = 0; i < k; ++i) {
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
    }

    cout << hungarian() << "\n";
```

```
    return 0;
}
```

图论

拓扑排序

拓扑排序 — Kahn 算法（入度 + 队列），附是否成环判定

```cpp
// 拓扑排序 — Kahn 算法（入度 + 队列），附是否成环判定

#include <bits/stdc++.h>
using namespace std;

/*
 * Kahn 拓扑排序：
 *   每次取出一个入度为 0 的点加入拓扑序列，并把它的所有出边"删掉"
 *   （即对应终点入度 -1），若某点入度归零则再入队。
 *   若最终能取出的点数 < n，说明图中存在环，无法拓扑排序。
 *   复杂度 O(n + m)。适用于有向无环图 (DAG)。
 */

const int MAXN = 1005;

vector<int> G[MAXN];   // 邻接表（存出边终点）

int indeg[MAXN];       // indeg[i] = 点 i 的入度


// 返回拓扑序列；若存在环则 vector 为空。

// n: 点数。

vector<int> topoSort(int n) {
    vector<int> ans;
    queue<int> q;

    for (int i = 1; i <= n; ++i)
        if (indeg[i] == 0) q.push(i); // 先把所有入度为 0 的点入队


    while (!q.empty()) {
        int u = q.front(); q.pop();
        ans.push_back(u);
        for (int v : G[u]) {
            if (--indeg[v] == 0) q.push(v); // 删边后入度归零则入队

        }
    }

    // 若取出的点数不足 n，说明有环

    if ((int)ans.size() != n) return {}; // 返回空表示有环

    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：一张有向无环图（DAG），输出其拓扑序列

    // 输入：第一行 n m（点数、边数），接下来 m 行 u v（u->v）

    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
```

```
        ++indeg[v]; // v 的入度 +1

    }

    vector<int> ans = topoSort(n);
    if (ans.empty()) {
        cout << "该图存在环，无法拓扑排序\n";
    } else {
        cout << "拓扑序列：";
        for (int i = 0; i < (int)ans.size(); ++i)
            cout << (i ? " " : "") << ans[i];
        cout << "\n";
    }
    return 0;
}
```

图论

网络流_Dinic

网络最大流 — Dinic（分层图 + 多路增广）

```cpp
// 网络最大流 — Dinic（分层图 + 多路增广）
#include <bits/stdc++.h>
using namespace std;

/*
 * Dinic 求最大流：
 *   1) BFS 建立分层图（dep[v] = 从 S 到 v 的最短边数）
 *   2) DFS 沿分层图一次找多条增广路，借助当前弧 cur[] 优化
 *   3) 重复上面两步，直到 BFS 到不了 T
 *
 *   复杂度 O(V^2 E)，实际常数小、性能很好，通常是网络流首选。
 *
 *   本模板用 P3376 的输入格式：用 long long 存容量。
 *   二分图最大匹配建模：S 连所有左部点(cap=1)，左部点连右部点(cap=1)，
 *   右部点连 T(cap=1)，答案即为最大流。
 */

typedef long long ll;
const int MAXN = 10005;
const ll INF = 0x3f3f3f3f3f3f3f3fll;

struct Edge { int to; ll cap; int rev; };
vector<Edge> G[MAXN];

int dep[MAXN], cur[MAXN]; // dep=分层深度, cur=当前弧

// 加一条有向边 u->v，并补上反向边（用于"退流"）
void addEdge(int u, int v, ll cap) {
    G[u].push_back({v, cap, (int)G[v].size()});
    G[v].push_back({u, 0,   (int)G[u].size() - 1});
}

// BFS 建分层图，返回能否到达汇点 T
bool bfs(int s, int t) {
    memset(dep, -1, sizeof(dep));
    queue<int> q;
    dep[s] = 0;
    q.push(s);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto &e : G[u]) {
            if (e.cap > 0 && dep[e.to] < 0) {
                dep[e.to] = dep[u] + 1;
                q.push(e.to);
            }
        }
    }
    return dep[t] >= 0;
}

// DFS 多路增广，返回本次增广的流量；f 为流入 u 的限制
ll dfs(int u, int t, ll f) {
    if (u == t) return f;
    for (int &i = cur[u]; i < (int)G[u].size(); ++i) {
        Edge &e = G[u][i];
        if (e.cap > 0 && dep[e.to] == dep[u] + 1) {
            ll d = dfs(e.to, t, min(f, e.cap));
            if (d > 0) {
                e.cap -= d;
                G[e.to][e.rev].cap += d; // 反向边增加，可"退流"
                return d;
            }
```

```
        }
    }
    return 0;
}

ll dinic(int s, int t) {
    ll flow = 0;
    while (bfs(s, t)) {
        memset(cur, 0, sizeof(cur));
        ll f;
        while ((f = dfs(s, t, INF)) > 0) flow += f;
    }
    return flow;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // P3376 输入：n m s t，接下来 m 行 u v c
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    for (int i = 0; i < m; ++i) {
        int u, v;
        ll c;
        cin >> u >> v >> c;
        addEdge(u, v, c);
    }

    cout << dinic(s, t) << "\n";
    return 0;
}
```

图论

Dijkstra

Dijkstra — 堆优化 Dijkstra（非负权单源最短路）：dist 数组 + 优先队列

```cpp
// Dijkstra — 堆优化 Dijkstra（非负权单源最短路）：dist 数组 + 优先队列

#include <bits/stdc++.h>
using namespace std;

/*
 * Dijkstra 算法：
 *   适用于【边权非负】的单源最短路。本质是贪心 + 松弛。
 *   用优先队列（小根堆）每次取出当前距离最小的未确定点来扩展，
 *   复杂度 O((n+m) log n)。
 * 注意：如果存在负权边，贪心会出错，应改用 SPFA / Bellman-Ford。
 */

typedef long long ll;
const int MAXN = 1005;
const ll INF = 0x3f3f3f3f3f3f3f3f; // 足够大的"无穷大"


struct Edge { int to; int w; };              // 目标点 + 边权

vector<Edge> G[MAXN];                        // 邻接表存图

ll dist[MAXN];                               // dist[i] = 源点到 i 的最短距离

bool vis[MAXN];                              // vis[i] = 点 i 是否已确定最短路


// s: 源点, n: 点数。跑完 dist[1..n] 即为答案，不可达的点保持 INF。

void dijkstra(int s, int n) {
    // pair<距离, 点>，默认按 first 从小到大 -> 小根堆

    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>> pq;

    for (int i = 1; i <= n; ++i) { dist[i] = INF; vis[i] = false; }
    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (vis[u]) continue;   // 这个点已经确定了最短路，跳过

        vis[u] = true;

        // 松弛 u 的所有出边

        for (auto &e : G[u]) {
            int v = e.to;
            // 通过 u 走到 v 更短 -> 更新

            if (dist[v] > dist[u] + e.w) {
                dist[v] = dist[u] + e.w;
                pq.push({dist[v], v});
            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
```

```cpp
    // 演示：一张小图，求 1 号点到其它所有点的最短路

    // 输入：第一行 n m（点数、边数），接下来 m 行 u v w

    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        G[u].push_back({v, w}); // 有向边

    }

    dijkstra(1, n);

    cout << "从 1 到各点的最短路：\n";
    for (int i = 1; i <= n; ++i) {
        cout << "  dist[" << i << "] = ";
        if (dist[i] == INF) cout << "INF\n";
        else cout << dist[i] << "\n";
    }
    return 0;
}
```

图论

Floyd

Floyd — Floyd-Warshall 全源最短路（O(n^3)）

```cpp
// Floyd — Floyd-Warshall 全源最短路（O(n^3)）

#include <bits/stdc++.h>
using namespace std;

/*
 * Floyd-Warshall 算法：
 *   求任意两点间的最短路（全源最短路），支持负权边（但不能有负环）。
 *   核心思想：dp[k][i][j] = 只经过前 k 个点中转时 i->j 的最短路。
 *   滚动掉第一维后即经典的 d[i][j] = min(d[i][j], d[i][k] + d[k][j])。
 *   复杂度 O(n^3)，通常 n <= 500 时可用；n 大时改用 Dijkstra(n 次) 或 Johnson。
 */

const int MAXN = 505;
const int INF = 0x3f3f3f3f;  // 足够大的"无穷大"，保证两倍仍不溢出


int d[MAXN][MAXN];           // d[i][j] = i 到 j 的最短路（也是距离矩阵）


// n: 点数。调用前需先初始化 d（见 main），之后 floyd 求出全源最短路。

void floyd(int n) {
    // 第一维滚动：先枚举中转点 k，再枚举 i、j

    for (int k = 1; k <= n; ++k)
        for (int i = 1; i <= n; ++i)
            for (int j = 1; j <= n; ++j)
                if (d[i][k] + d[k][j] < d[i][j])
                    d[i][j] = d[i][k] + d[k][j];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：一张带权小图，用距离矩阵初始化并求全源最短路

    // 输入：第一行 n m（点数、边数），接下来 m 行 u v w

    // 不存在的边用 INF 表示；自己到自己为 0

    int n, m;
    cin >> n >> m;

    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j)
            d[i][j] = (i == j ? 0 : INF);  // 对角为 0，其余为 INF


    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        d[u][v] = min(d[u][v], w);  // 有向边，取最短以防重边

    }

    floyd(n);

    cout << "全源最短路（距离矩阵）：\n";
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
```

```
            if (i == j) cout << "0 ";
            else if (d[i][j] == INF) cout << "INF ";
            else cout << d[i][j] << " ";
        }
        cout << "\n";
    }
    return 0;
}
```

图论

Kruskal

Kruskal — 最小生成树 Kruskal（边按权排序 + 并查集）

```cpp
// Kruskal — 最小生成树 Kruskal（边按权排序 + 并查集）

#include <bits/stdc++.h>
using namespace std;

/*
 * Kruskal 最小生成树：
 *   把所有边按权值从小到大排序，依次考虑每条边 (u,v,w)，
 *   若 u、v 尚未连通（不在同一并查集），则选入生成树并合并。
 *   选到 n-1 条边即得到最小生成树；若遍历完仍不足 n-1 条，说明图不连通。
 *   复杂度 O(m log m)（主要是排序）。适用于求边权和最小的生成树。
 *   并查集用于动态判断两点是否连通，接近 O(1)。
 */

const int MAXN = 1005;

struct Edge {
    int u, v, w;
    // 按权值从小到大排序

    bool operator<(const Edge &o) const { return w < o.w; }
};
vector<Edge> edges; // 所有边


int fa[MAXN]; // 并查集父节点


// 并查集初始化

void initDSU(int n) { for (int i = 1; i <= n; ++i) fa[i] = i; }

// 查找根 + 路径压缩

int find(int x) { return fa[x] == x ? x : (fa[x] = find(fa[x])); }

// 合并两集合，成功返回 true，已在同一集合返回 false

bool unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return false;
    fa[rx] = ry;
    return true;
}

// n: 点数。返回最小生成树边权和；若图不连通返回 -1。

long long kruskal(int n) {
    initDSU(n);
    sort(edges.begin(), edges.end()); // 边按权排序


    long long sum = 0;
    int cnt = 0; // 已选入的边数

    for (auto &e : edges) {
        if (unite(e.u, e.v)) {
            sum += e.w;
            if (++cnt == n - 1) break; // 选够了 n-1 条边

        }
    }
```

```cpp
    if (cnt < n - 1) return -1; // 不连通，不存在生成树

    return sum;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：一张连通带权小图，求最小生成树边权和

    // 输入：第一行 n m（点数、边数），接下来 m 行 u v w（无向边）

    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        edges.push_back({u, v, w});
    }

    long long sum = kruskal(n);
    if (sum == -1) cout << "图不连通，没有最小生成树\n";
    else cout << "最小生成树边权和 = " << sum << "\n";
    return 0;
}
```

图论

LCA

LCA — 树上最近公共祖先，倍增思想（depth、up[N][LOG]）

```cpp
// LCA — 树上最近公共祖先，倍增思想（depth、up[N][LOG]）

#include <bits/stdc++.h>
using namespace std;

/*
 * 倍增法求 LCA（最近公共祖先）：
 *   预处理：一次 DFS/BFS 求出每个点的深度 depth[u]，
 *          以及 up[u][k] = 从 u 向上跳 2^k 步到达的祖先。
 *   查询 lca(u,v)：
 *       1) 先把较深的点向上跳到与另一个同深度；
 *       2) 再从大到小尝试一起向上跳，若跳 2^k 步后不相等则跳；
 *       3) 最终父节点即为 LCA。
 *   预处理 O(n log n)，每次查询 O(log n)。
 */

const int MAXN = 1005;
const int LOG = 11; // > log2(最大深度)，对 n<=1000 取 11 足够


vector<int> G[MAXN];   // 邻接表（存子结点）

int depth[MAXN];       // depth[u] = u 的深度（根为 1）

int up[MAXN][LOG];     // up[u][k] = u 向上 2^k 步的祖先


// 从根开始 DFS 预处理 depth 与 up

void dfs(int u, int parent) {
    up[u][0] = parent;
    for (int k = 1; k < LOG; ++k) up[u][k] = up[up[u][k - 1]][k - 1];

    for (int v : G[u]) {
        if (v == parent) continue; // 不再走回父结点

        depth[v] = depth[u] + 1;
        dfs(v, u);
    }
}

// 查询 u, v 的最近公共祖先

int lca(int u, int v) {
    // 1) 让 u 是较深的那个

    if (depth[u] < depth[v]) swap(u, v);

    // 2) 把 u 上跳到和 v 同深度

    int diff = depth[u] - depth[v];
    for (int k = 0; k < LOG; ++k)
        if (diff >> k & 1) u = up[u][k];

    // 已经相遇

    if (u == v) return u;

    // 3) 一起向上跳，跳到 LCA 正下方

    for (int k = LOG - 1; k >= 0; --k) {
        if (up[u][k] != up[v][k]) {
```

```
            u = up[u][k];
            v = up[v][k];
        }
    }
    return up[u][0]; // 父结点即 LCA

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：一棵小树，求若干对点的 LCA

    // 输入：第一行 n（点数，根为 1），接下来 n-1 行 u v 表示一条树边

    //       之后若干组查询，每行两个点，直到文件结束：求其 LCA

    int n;
    cin >> n;
    for (int i = 0; i < n - 1; ++i) {
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
        G[v].push_back(u);
    }

    depth[1] = 0;
    dfs(1, 0);

    int u, v;
    while (cin >> u >> v) {
        cout << "LCA(" << u << ", " << v << ") = " << lca(u, v) << "\n";
    }
    return 0;
}
```

图论

SPFA

SPFA — 队列优化 Bellman-Ford，可判负环（cnt 记录入队次数）

```cpp
// SPFA — 队列优化 Bellman-Ford，可判负环（cnt 记录入队次数）

#include <bits/stdc++.h>
using namespace std;

/*
 * SPFA（Shortest Path Faster Algorithm）：
 *   Bellman-Ford 的队列优化版，求单源最短路。
 *   松弛一条边后，若终点 dist 变小则把它加入队列等待再次松弛。
 *   判断负环：记录每个点入队次数 cnt[v]，若某个点入队次数 >= n，
 *   说明图中存在从源点可达的负环（此时没有最短路）。
 *   复杂度理论上界 O(nm)，实际通常很快，但在构造数据下可能退化。
 */

const int MAXN = 1005;
const int INF = 0x3f3f3f3f;

struct Edge { int to; int w; };
vector<Edge> G[MAXN];

int dist[MAXN];
int cnt[MAXN];      // cnt[i] = i 入队次数，用于判负环

bool inQueue[MAXN]; // inQueue[i] = i 是否在队列中


// 返回 true 表示存在负环，false 表示正常求出了最短路。

// 结果保存在 dist[1..n]。

bool spfa(int s, int n) {
    queue<int> q;
    for (int i = 1; i <= n; ++i) { dist[i] = INF; cnt[i] = 0; inQueue[i] = false; }

    dist[s] = 0;
    q.push(s);
    inQueue[s] = true;
    cnt[s] = 1;

    while (!q.empty()) {
        int u = q.front(); q.pop();
        inQueue[u] = false;

        for (auto &e : G[u]) {
            int v = e.to;
            if (dist[v] > dist[u] + e.w) {   // 松弛成功

                dist[v] = dist[u] + e.w;
                if (!inQueue[v]) {
                    q.push(v);
                    inQueue[v] = true;
                    if (++cnt[v] >= n) return true; // 入队达到 n 次 => 存在负环

                }
            }
        }
    }
    return false; // 没有负环

}

int main() {
```

```cpp
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：跑 SPFA 求最短路，并可选地检测负环

    // 输入：第一行 n m（点数、边数），接下来 m 行 u v w

    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        G[u].push_back({v, w});
    }

    bool negCycle = spfa(1, n);
    if (negCycle) {
        cout << "存在从 1 可达的负环，无最短路\n";
    } else {
        cout << "从 1 到各点的最短路：\n";
        for (int i = 1; i <= n; ++i) {
            cout << "  dist[" << i << "] = ";
            if (dist[i] == INF) cout << "INF\n";
            else cout << dist[i] << "\n";
        }
    }
    return 0;
}
```

图论 Tarjan强连通分量

Tarjan强连通分量 — Tarjan 求强连通分量（dfn、low、栈、scc 计数）

```cpp
// Tarjan强连通分量 — Tarjan 求强连通分量（dfn、low、栈、scc 计数）

#include <bits/stdc++.h>
using namespace std;

/*
 * Tarjan 求强连通分量 (SCC):
 *   dfn[u] = u 第一次被访问的时间戳（dfs 序）。
 *   low[u] = u 及其子孙能回溯到的最小 dfn。
 *   在 DFS 过程中用一个栈维护"尚未确定属于哪一个 SCC"的点。
 *   当 low[u] == dfn[u] 时，u 是一个强连通分量的根，
 *   把栈顶一直弹到 u，这些点构成一个 SCC。
 *   复杂度 O(n + m)。适用于有向图。
 */

const int MAXN = 1005;

vector<int> G[MAXN];
int dfn[MAXN];     // 时间戳

int low[MAXN];     // 能回溯到的最小 dfn

int scc[MAXN];     // scc[i] = 点 i 所属的 SCC 编号（从 1 开始）

bool inStack[MAXN]; // 是否在栈中

int timer = 0;      // 全局时间戳计数

int sccCnt = 0;     // SCC 总数

stack<int> st;

void tarjan(int u) {
    dfn[u] = low[u] = ++timer;
    st.push(u);
    inStack[u] = true;

    for (int v : G[u]) {
        if (!dfn[v]) {          // v 尚未访问 -> 递归

            tarjan(v);
            low[u] = min(low[u], low[v]); // 回溯时更新 low

        } else if (inStack[v]) { // v 已在栈中 -> 是回边/横叉边

            low[u] = min(low[u], dfn[v]);
        }
    }

    // 若 u 是所在 SCC 的根，则弹栈收集整个分量

    if (low[u] == dfn[u]) {
        ++sccCnt;
        while (true) {
            int x = st.top(); st.pop();
            inStack[x] = false;
            scc[x] = sccCnt; // 标记归属

            if (x == u) break;
        }
    }
}
```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：一张小有向图，输出每个强连通分量包含的点

    // 输入：第一行 n m（点数、边数），接下来 m 行 u v（u->v）

    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        G[u].push_back(v);
    }

    // 图可能不连通，对所有未访问的点都跑一次 tarjan

    for (int i = 1; i <= n; ++i)
        if (!dfn[i]) tarjan(i);

    cout << "共有 " << sccCnt << " 个强连通分量：\n";
    // 按分量编号归类输出

    vector<vector<int>> comp(sccCnt + 1);
    for (int i = 1; i <= n; ++i) comp[scc[i]].push_back(i);
    for (int c = 1; c <= sccCnt; ++c) {
        cout << "  SCC" << c << ":";
        for (int x : comp[c]) cout << " " << x;
        cout << "\n";
    }
    return 0;
}
```

字符串

字符串

字符串哈希

字符串哈希 — 滚动哈希（无符号自然溢出），快速判断两个子串是否相等

```cpp
// 字符串哈希 — 滚动哈希（无符号自然溢出），快速判断两个子串是否相等

#include <bits/stdc++.h>
using namespace std;

typedef unsigned long long ull;

// 字符串哈希结构体。

// 原理：把字符串看作 base 进制的大整数；这里用 unsigned long long 自然溢出，

// 相当于自动对 2^64 取模，避免手动写模运算。由前缀哈希 h[i] 可 O(1) 求任意子串 [l, r) 的哈希。

// 若想进一步降低冲突概率，可额外用一个大质数（如 1e9+7）做第二哈希。

struct StrHash {
    int n;
    vector<ull> h;   // h[i]：s[0..i-1] 的哈希

    vector<ull> pw;  // pw[i]：base 的 i 次方

    StrHash(const string& s, ull base) {
        n = (int)s.size();
        h.assign(n + 1, 0);
        pw.assign(n + 1, 1);
        for (int i = 0; i < n; ++i) {
            h[i + 1] = h[i] * base + (ull)(s[i] - 'a' + 1); // 字符映射为 1..26

            pw[i + 1] = pw[i] * base;
        }
    }
    // 返回子串 s[l..r) 的哈希

    ull get(int l, int r) const {
        return h[r] - h[l] * pw[r - l];
    }
    // 判断两个子串 s[l1..r1) 与 s[l2..r2) 是否相等

    bool equal(int l1, int r1, int l2, int r2) const {
        if (r1 - l1 != r2 - l2) return false; // 长度不同必然不等

        return get(l1, r1) == get(l2, r2);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    string s = "banana"; // 演示：比较字符串中两个子串是否相等

    string base = "abcdefghijklmnopqrstuvwxyz";
    cout << "字符串: " << s << "\n";

    const ull BASE = 131;          // 常用进制

    StrHash hash(s, BASE);

    // 比较 s[0..2) = "ban" 与 s[3..6) = "ana"（应不等）

    cout << "子串[0,3)=\"" << s.substr(0, 3) << "\" 与 [3,6)=\"" << s.substr(3, 3)
         << "\" 是否相等: " << (hash.equal(0, 3, 3, 6) ? "是" : "否") << "\n";
```

```
    // 比较 s[1..4) = "ana" 与 s[3..6) = "ana"（应相等）

    cout << "子串[1,4)=\"" << s.substr(1, 3) << "\" 与 [3,6)=\"" << s.substr(3, 3)
         << "\" 是否相等: " << (hash.equal(1, 4, 3, 6) ? "是" : "否") << "\n";
    return 0;
}
```

字符串

KMP

KMP — 前缀函数 next/pi + KMP 匹配，在文本串中找模式串的所有出现位置

```cpp
// KMP — 前缀函数 next/pi + KMP 匹配，在文本串中找模式串的所有出现位置

#include <bits/stdc++.h>
using namespace std;

// 求模式串 p 的前缀函数（也叫 next 数组）。

// pi[i] 表示子串 p[0..i] 的最长「真前缀」同时也是「真后缀」的长度。

// 复杂度：O(|p|)

vector<int> get_pi(const string& p) {
    int n = (int)p.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];          // 继承上一个位置的匹配长度

        while (j > 0 && p[i] != p[j]) j = pi[j - 1]; // 失配则回退到上一个可匹配的前缀

        if (p[i] == p[j]) ++j;      // 能匹配，长度加 1

        pi[i] = j;                  // 记录当前最长公共前后缀

    }
    return pi;
}

// KMP 匹配：在文本串 t 中查找模式串 p 的所有出现位置（0 基下标）。

// 复杂度：O(|t| + |p|)

vector<int> kmp_match(const string& t, const string& p) {
    vector<int> pi = get_pi(p);
    vector<int> ans;
    int j = 0;                      // 当前已匹配的模式串长度

    for (int i = 0; i < (int)t.size(); ++i) {
        while (j > 0 && t[i] != p[j]) j = pi[j - 1]; // 失配回退

        if (t[i] == p[j]) ++j;      // 匹配成功一个字符

        if (j == (int)p.size()) {   // 匹配到完整模式串

            ans.push_back(i - j + 1); // 记录起始下标

            j = pi[j - 1];          // 回退以寻找重叠出现的模式串

        }
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    string t = "abababcabab", p = "aba"; // 演示：文本串与模式串

    cout << "文本串: " << t << "\n模式串: " << p << "\n";
    cout << "前缀函数 pi: ";
    for (int v : get_pi(p)) cout << v << ' ';
    cout << "\n模式串出现的位置(0基): ";
```

```
    vector<int> ans = kmp_match(t, p);
    if (ans.empty()) cout << "无";
    for (int pos : ans) cout << pos << ' ';
    cout << "\n";
    return 0;
}
```

字符串

Manacher

Manacher — 马拉车算法求所有回文子串半径，用于求最长回文子串

```cpp
// Manacher — 马拉车算法求所有回文子串半径，用于求最长回文子串

#include <bits/stdc++.h>
using namespace std;

// 马拉车算法。

// d1[i]：以 i 为中心、长度为奇数的最长回文半径（含中心），即回文串是 s[i-d1[i]+1 .. i+d1[i]-1]。

// d2[i]：以 s[i-1] 与 s[i] 之间为中心、长度为偶数的最长回文半径，即回文串是 s[i-d2[i] .. i+d2[i]-1]。

// 复杂度：O(n)，每个字符最多被扩展一次。

void manacher(const string& s, vector<int>& d1, vector<int>& d2) {
    int n = (int)s.size();
    d1.assign(n, 0);
    d2.assign(n, 0);

    // 处理奇数长度的回文

    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1); // 利用已有信息初始化长度

        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) ++k; // 向外扩展

        d1[i] = k;
        if (i + k - 1 > r) { l = i - k + 1; r = i + k - 1; }          // 更新最右回文区间

    }

    // 处理偶数长度的回文

    for (int i = 0, l = 0, r = -1; i < n; ++i) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1); // 初始化长度

        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) ++k; // 向外扩展

        d2[i] = k;
        if (i + k - 1 > r) { l = i - k; r = i + k - 1; }               // 更新最右回文区间

    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    string s = "aabacacaa"; // 演示：计算最长回文子串长度

    cout << "字符串: " << s << "\n";

    vector<int> d1, d2;
    manacher(s, d1, d2);

    int best = 1; // 最长回文长度至少为 1（单个字符）

    for (int i = 0; i < (int)s.size(); ++i) {
        best = max(best, 2 * d1[i] - 1); // 奇数回文：长度 = 2*d1 - 1

        best = max(best, 2 * d2[i]);     // 偶数回文：长度 = 2*d2

    }
```

```
    cout << "最长回文子串长度: " << best << "\n";
    return 0;
}
```

搜索

搜索 0-1BFS

0-1 BFS — 双端队列求边权仅为 0 或 1 的最短路（复杂度 O(V+E)）

```cpp
// 0-1 BFS — 双端队列求边权仅为 0 或 1 的最短路（复杂度 O(V+E)）

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
const int INF = 0x3f3f3f3f;

vector<pair<int, int>> g[MAXN]; // 邻接表：g[u] = {v, w}，边权 w 为 0 或 1

int dist[MAXN];                 // 起点到各点的最短距离


// 0-1 BFS：从 s 出发求到各点的最短距离。

// 关键：因为边权只有 0/1，用双端队列维护——松弛到 0 边放队首，1 边放队尾，

// 保证队列始终按距离非降序，每个点被正确更新且复杂度 O(V+E)。

void bfs01(int s, int n) {
    for (int i = 1; i <= n; ++i) dist[i] = INF;
    deque<int> dq;
    dist[s] = 0;
    dq.push_back(s);
    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();
        for (auto& e : g[u]) {
            int v = e.first, w = e.second;
            if (dist[u] + w < dist[v]) {   // 松弛成功

                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v); // 0 边放队首（距离不加）

                else dq.push_back(v);         // 1 边放队尾

            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：5 个点，边权为 0 或 1 的有向小图

    int n = 5;
    g[1].push_back({2, 0});
    g[1].push_back({3, 1});
    g[2].push_back({3, 0});
    g[2].push_back({4, 1});
    g[3].push_back({4, 0});
    g[3].push_back({5, 1});
    g[4].push_back({5, 0});

    bfs01(1, n); // 从点 1 出发

    cout << "从点 1 出发到各点的最短距离:\n";
    for (int i = 1; i <= n; ++i)
        cout << "到点 " << i << ": " << (dist[i] == INF ? "INF" : to_string(dist[i])) << "\n";
```

```
    return 0;
}
```

搜索 记忆化搜索

记忆化搜索 — 带 memo 数组的递归，消除重复子问题（以斐波那契为例）

```cpp
// 记忆化搜索 — 带 memo 数组的递归，消除重复子问题（以斐波那契为例）

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

const int MAXN = 100;
ll memo[MAXN];     // memo[i]：已算好的 f(i)，-1 表示尚未计算

bool computed[MAXN]; // 标记 f(i) 是否已计算（也可直接用 memo 初值判空）


// 记忆化求斐波那契第 n 项：f(0)=0, f(1)=1, f(n)=f(n-1)+f(n-2)。

// 朴素递归会指数级重复计算；记忆化后每个状态只算一次，复杂度 O(n)。

ll fib(int n) {
    if (n <= 1) return n;           // 边界：f(0)=0, f(1)=1

    if (computed[n]) return memo[n]; // 已算过，直接返回（这就是记忆化的核心）

    memo[n] = fib(n - 1) + fib(n - 2); // 计算并记录

    computed[n] = true;
    return memo[n];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：计算斐波那契第 n 项并说明记忆化如何避免重复

    memset(computed, false, sizeof(computed));
    int n = 30;
    cout << "计算斐波那契第 " << n << " 项\n";
    cout << "fib(" << n << ") = " << fib(n) << "\n";
    cout << "说明：朴素递归会重复计算大量相同的子问题（如 fib(28)、fib(29) 各被调用多次），"
            "而记忆化搜索把每个结果存入 memo，之后遇到同样参数直接用，"
            "因此总计算量从指数级降到 O(n)。\n";
    return 0;
}
```

搜索 **BFS**

BFS — 网格广度优先搜索，求起点到终点的最短步数（'.' 为通道，'#' 为障碍）

```cpp
// BFS — 网格广度优先搜索，求起点到终点的最短步数（'.' 为通道，'#' 为障碍）

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 105;
int n, m;                                // 网格行数、列数

char grid[MAXN][MAXN];                // 网格

int dist[MAXN][MAXN];                 // 到起点的最短步数，-1 表示未访问

const int dx[4] = {-1, 1, 0, 0};      // 上下左右

const int dy[4] = {0, 0, -1, 1};      // 上下左右


// BFS：从 (sx, sy) 出发求到 (ex, ey) 的最短步数，不可达返回 -1。

// 网格无边权且每步代价为 1，BFS 按层扩展，首次到达即为最短。

int bfs(int sx, int sy, int ex, int ey) {
    memset(dist, -1, sizeof(dist));
    queue<pair<int, int>> q;
    dist[sx][sy] = 0;
    q.push({sx, sy});
    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        if (x == ex && y == ey) return dist[x][y]; // 到达终点

        for (int k = 0; k < 4; ++k) {              // 尝试四个方向

            int nx = x + dx[k], ny = y + dy[k];
            // 越界、撞墙、已访问的都跳过

            if (nx < 0 || nx >= n || ny < 0 || ny >= m) continue;
            if (grid[nx][ny] == '#') continue;
            if (dist[nx][ny] != -1) continue;
            dist[nx][ny] = dist[x][y] + 1;         // 步数 +1

            q.push({nx, ny});
        }
    }
    return -1; // 无法到达

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：5x5 小网格，起点 (0,0)，终点 (4,4)

    n = 5; m = 5;
    vector<string> g = {
        ". . . # .",
        "# . . . .",
        ". # . # .",
        ". . . . .",
        ". # . . ."
    };
```

```
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < m; ++j)
            grid[i][j] = g[i][j * 2]; // 每行以空格分隔，取第 2j 个字符


    cout << "网格(" << n << "x" << m << "):\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) cout << grid[i][j] << ' ';
        cout << "\n";
    }
    int step = bfs(0, 0, 4, 4);
    if (step == -1) cout << "起点无法到达终点\n";
    else cout << "起点(0,0)到终点(4,4)最短步数: " << step << "\n";
    return 0;
}
```

搜索 **DFS**

DFS — 深度优先搜索回溯模板，以全排列为例（idx + vis + 回溯）

```cpp
// DFS — 深度优先搜索回溯模板，以全排列为例（idx + vis + 回溯）

#include <bits/stdc++.h>
using namespace std;

int n;                      // 要排列的元素个数

vector<int> path;           // 当前排好的序列（全局变量，避免栈爆）

bool vis[20];               // vis[i] 标记数字 i 是否已被使用


// 递归回溯函数：正在确定排列的第 step 个位置（0 基）。

// 复杂度：O(n!)（枚举了所有排列）

void dfs(int step) {
    if (step == n) {        // 已填满 n 个位置，输出一个完整排列

        for (int i = 0; i < n; ++i) cout << path[i] << (i + 1 == n ? "\n" : " ");
        return;
    }
    for (int i = 1; i <= n; ++i) { // 尝试把数字 i 放到当前位

        if (vis[i]) continue;      // 该数字已被使用，跳过

        vis[i] = true;             // 标记使用

        path.push_back(i);         // 填入当前位

        dfs(step + 1);             // 递归确定下一个位置

        path.pop_back();           // 回溯：撤销填入

        vis[i] = false;            // 回溯：撤销标记

    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    n = 3; // 演示：输出 [1,2,3] 的全排列

    cout << "1.." << n << " 的全排列:\n";
    dfs(0);
    return 0;
}
```

DP

DP 0-1背包

0-1背包 — 0-1 背包：一维滚动数组（体积逆序）求最大价值

```cpp
// 0-1背包 — 0-1 背包：一维滚动数组（体积逆序）求最大价值

#include <bits/stdc++.h>
using namespace std;

// 物品数上限与体积上限（可按题目调整）

const int MAXN = 105;   // 物品个数上限

const int MAXV = 1005;  // 背包容量上限


int v[MAXN];  // v[i]：第 i 件物品的体积

int w[MAXN];  // w[i]：第 i 件物品的价值

int dp[MAXV]; // dp[c]：容量为 c 时能获得的最大价值


/*
 * 0-1 背包（一维滚动数组优化）
 * 状态定义：dp[c] 表示背包容量为 c 时能装下的最大价值。
 * 转移方程（对第 i 件物品，容量逆序更新）：
 *   dp[c] = max(dp[c], dp[c - v[i]] + w[i]),  c: V -> v[i]
 * 逆序更新保证每件物品最多只被选一次（0-1 特性）。
 * 初始：dp[0..V] = 0。
 * 复杂度：O(n * V)，空间 O(V)。
 */
int knap01(int n, int V) {
    for (int i = 1; i <= n; ++i)
        for (int c = V; c >= v[i]; --c)   // 体积逆序

            dp[c] = max(dp[c], dp[c - v[i]] + w[i]);
    // dp[V] 即为容量正好不超过 V 时的最大价值

    return *max_element(dp, dp + V + 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：4 件物品，容量 8 的背包，求最大价值

    int n = 4, V = 8;
    int vv[] = {2, 3, 4, 5};
    int ww[] = {3, 4, 5, 6};
    for (int i = 1; i <= n; ++i) { v[i] = vv[i - 1]; w[i] = ww[i - 1]; }

    cout << "0-1 背包最大价值：" << knap01(n, V) << '\n';
    // 输出每个容量下的最优价值（滚动数组最终值）

    for (int c = 1; c <= V; ++c)
        cout << "容量" << c << "=" << dp[c] << (c == V ? '\n' : ' ');
    return 0;
}
```

DP 多重背包

多重背包 — 多重背包：二进制拆分转化为 0-1 背包求最大价值

```cpp
// 多重背包 — 多重背包：二进制拆分转化为 0-1 背包求最大价值

#include <bits/stdc++.h>
using namespace std;

// 体积上限与拆分后物品数上限

const int MAXN = 105;
const int MAXV = 2005;
const int MAXK = MAXN * 20; // 拆分后物品总数（原物品数 * log 数量）


struct Good { int v, w; } a[MAXK]; // 拆分后的每个"新物品"

int dp[MAXV];
int cnt = 0; // 拆分后的物品件数


/*
 * 多重背包：每种物品有数量限制 num。
 * 二进制拆分思想：把 num 拆成 1, 2, 4, ..., 2^k 以及剩余 r 若干组，
 * 每组作为一个"新物品"（体积 = 组数 * 原体积，价值 = 组数 * 原价值）。
 * 这样 0..num 的任意取法都能由若干组组合出来，且每组最多取一次，
 * 于是原多重背包问题等价于 0-1 背包。
 * 拆分复杂度 O(log num)，总复杂度 O(n * log num * V)。
 */
void addItem(int v, int w, int num) {
    for (int k = 1; num > 0; k <<= 1) {
        int take = min(k, num);
        a[++cnt] = {take * v, take * w};
        num -= take;
    }
}

// 对拆分后得到的 cnt 个 0-1 物品做 0-1 背包（体积逆序）

int knapMultiple(int n, int V) {
    for (int i = 1; i <= cnt; ++i)
        for (int c = V; c >= a[i].v; --c)
            dp[c] = max(dp[c], dp[c - a[i].v] + a[i].w);
    return dp[V];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
    memset(dp, 0, sizeof(dp));
    cnt = 0;

    // 演示：4 种物品（体积、价值、数量），背包容量 10

    int n = 4, V = 10;
    int vv[] = {2, 3, 4, 5};
    int ww[] = {3, 4, 5, 6};
    int cc[] = {3, 2, 1, 2};
    for (int i = 1; i <= n; ++i)
        addItem(vv[i - 1], ww[i - 1], cc[i - 1]);

    cout << "多重背包最大价值：" << knapMultiple(n, V) << '\n';
    return 0;
}
```

DP 区间DP

区间DP — 区间 DP：石子合并（线性弱化版）求最小合并代价

```cpp
// 区间DP — 区间 DP：石子合并（线性弱化版）求最小合并代价

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 305;
const int INF = 0x3f3f3f3f;
int a[MAXN];       // a[i]：第 i 堆石子数

int sum[MAXN];     // 前缀和：sum[i] = a[1] + ... + a[i]

int f[MAXN][MAXN]; // f[l][r]：把第 l..r 堆合并成一堆的最小代价


/*
 * 区间 DP（石子合并，线性，只能合并相邻两堆）。
 * 状态：f[l][r] 表示把区间 [l, r] 的石子合并成一堆所需的最小代价。
 * 转移（枚举最后一步合并的分界点 k，k 属于 [l, r-1]）：
 *   f[l][r] = min_{k} ( f[l][k] + f[k+1][r] ) + sum[r] - sum[l-1]
 * 其中 sum[r] - sum[l-1] 为本区间石子总数，也就是最后一次合并的代价。
 * 边界：f[i][i] = 0（单堆已是一堆，无需代价）。
 * 枚举顺序：按区间长度 len 从小到大枚举（保证子区间先算好）。
 * 复杂度：O(n^3)，空间 O(n^2)。
 * 换成环形石子合并时，将数组复制一份拼接成长度 2n，做同样的线性合并，
 * 最后取 min_{i=1..n} f[i][i+n-1] 即可。
 */
int stoneMerge(int n) {
    // 前缀和

    for (int i = 1; i <= n; ++i) sum[i] = sum[i - 1] + a[i];
    // 初始化：单堆代价 0，其余为无穷大

    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= n; ++j) f[i][j] = (i == j) ? 0 : INF;

    for (int len = 2; len <= n; ++len) {          // 枚举区间长度

        for (int l = 1; l + len - 1 <= n; ++l) {  // 枚举左端点

            int r = l + len - 1;
            for (int k = l; k < r; ++k)           // 枚举分界点

                f[l][r] = min(f[l][r], f[l][k] + f[k + 1][r]);
            f[l][r] += sum[r] - sum[l - 1];       // 加上本次合并代价

        }
    }
    return f[1][n];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：4 堆石子，求最小合并代价

    int n = 4;
    int arr[] = {1, 2, 3, 4};
    for (int i = 1; i <= n; ++i) a[i] = arr[i - 1];

    cout << "最小合并代价：" << stoneMerge(n) << '\n';
```

```
    return 0;
}
```

DP 树形DP

树形DP — 树上最大独立集（没有上司的舞会）：f[u][0/1]

```cpp
// 树形DP — 树上最大独立集（没有上司的舞会）：f[u][0/1]

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 200005;
vector<int> g[MAXN]; // 邻接表存树（父 -> 子）

int val[MAXN];       // 每个结点的权值

int f[MAXN][2];      // f[u][0/1]：u 不选 / 选时，u 子树的最大权值和

bool hasFa[MAXN];    // 是否有父结点（用于找根）


/*
 * 树上最大独立集（相邻点不能同时选，经典"没有上司的舞会"）。
 * 状态定义：
 *   f[u][0] = u 不选时，以 u 为根的子树能取到的最大权值和；
 *   f[u][1] = u  选时，以 u 为根的子树能取到的最大权值和。
 * 转移（对每个子结点 v，后序遍历累加进来）：
 *   f[u][0] += max(f[v][0], f[v][1]);   // u 不选，子结点可选可不选
 *   f[u][1] += f[v][0];                 // u  选，子结点必须不选
 * 边界：叶子 f[u][0] = 0，f[u][1] = val[u]（初始化时令 f[u][1] = val[u]）。
 * 答案：max(f[root][0], f[root][1])。
 * 复杂度：O(n)。
 */
void dfs(int u) {
    f[u][1] = val[u];
    for (int v : g[u]) {
        dfs(v);
        f[u][0] += max(f[v][0], f[v][1]);
        f[u][1] += f[v][0];
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：7 个结点的树，求最大选点权值和（相邻不能同时选）

    int n = 7;
    int w[] = {0, 1, 2, 3, 4, 5, 6, 7}; // val[1..7] = 1..7

    for (int i = 1; i <= n; ++i) val[i] = w[i];

    // 建树（有向：父 -> 子）

    int edge[][2] = {{1, 2}, {1, 3}, {2, 4}, {2, 5}, {3, 6}, {3, 7}};
    for (int i = 0; i < n - 1; ++i) {
        int u = edge[i][0], v = edge[i][1];
        g[u].push_back(v);
        hasFa[v] = true;
    }
    int root = 1;
    for (int i = 1; i <= n; ++i)
        if (!hasFa[i]) { root = i; break; }

    dfs(root);
    cout << "最大选点权值和：" << max(f[root][0], f[root][1]) << '\n';
    // 如选 {1,4,5,6,7}: 1+4+5+6+7 = 23
```

```
    return 0;
}
```

DP 数位DP

数位DP — 数位 DP：按位处理 + 记忆化，统计 [0,x] 中满足条件的数的个数

```cpp
// 数位DP — 数位 DP：按位处理 + 记忆化，统计 [0,x] 中满足条件的数的个数

#include <bits/stdc++.h>
using namespace std;

int num[25];                            // 把上界 x 拆成十进制各低位（num[0] 为个位）

long long memo[25][2][2][25];           // memo[pos][limit][lead][cnt] 记忆化

// pos: 当前处理到的位；limit: 此前是否一直贴着上界；lead: 是否仍处前导零；cnt: 已累加的计数


/*
 * 数位 DP：统计 [0, x] 内所有数的十进制表示中数字 c 出现的总次数。
 * 从高位向低位递归，每一位枚举填 d：0..(limit ? 该位上界 : 9)。
 * 参数含义：
 *   pos    当前处理从最高位起的第 pos 位（数位从高到低）；
 *   limit  此前是否一直贴着上界，若是则当前位只能填到 num[pos]；
 *   lead   是否仍处于前导零阶段（之前全是 0）；
 *   cnt    之前已经出现的数字 c 的次数（作为 state）；
 *   c      要找的目标数字。
 * 递归返回：从当前位继续往后能贡献的 c 的总次数。
 * 记忆化：只有当 !limit && !lead 时状态才不受上界/前导零影响，可复用。
 * 复杂度：O(位数 * 2 * 2 * 位数)。
 * 若要统计"满足某条件（如含数字 X、可被某数整除、各位和等）的数的个数"，
 * 只需替换 state（cnt）与加贡献（add）的判定即可。
 */
long long dfs(int pos, int limit, int lead, int cnt, int c) {
    if (pos < 0) return cnt;                                    // 所有位处理完

    if (!limit && !lead && memo[pos][limit][lead][cnt] != -1)
        return memo[pos][limit][lead][cnt];

    int up = limit ? num[pos] : 9;
    long long res = 0;
    for (int d = 0; d <= up; ++d) {
        // 统计本位置为 d 时对数字 c 的贡献：

        // 处于前导零阶段且 d == 0 时不算作数字 0 的计数（常见约定）

        int add = (!(lead && d == 0) && d == c) ? 1 : 0;
        res += dfs(pos - 1, limit && d == up, lead && d == 0, cnt + add, c);
    }
    if (!limit && !lead) memo[pos][limit][lead][cnt] = res;
    return res;
}

// 统计 [0, x] 中数字 c 出现的总次数

long long countDigit(long long x, int c) {
    if (x < 0) return 0;
    int len = 0;
    while (x) { num[len++] = x % 10; x /= 10; }
    if (len == 0) num[len++] = 0;           // 特判 x == 0

    memset(memo, -1, sizeof(memo));
    return dfs(len - 1, 1, 1, 0, c);        // 从最高位开始

}

int main() {
    ios::sync_with_stdio(false);
```

```
    cin.tie(0);

    // 演示：统计 [0, 20] 内数字 1 出现的总次数

    long long ans = countDigit(20, 1);
    cout << "[0, 20] 中数字 1 出现的次数：" << ans << '\n';
    // 验证：1(1次), 10(1), 11(2), 12~19(各1,共8) -> 1+1+2+8 = 12

    return 0;
}
```

DP 完全背包

完全背包 — 完全背包：一维正序循环求最大价值

```cpp
// 完全背包 — 完全背包：一维正序循环求最大价值

#include <bits/stdc++.h>
using namespace std;

// 物品数上限与体积上限（可按题目调整）

const int MAXN = 105;
const int MAXV = 1005;

int v[MAXN];  // v[i]：第 i 种物品的体积

int w[MAXN];  // w[i]：第 i 种物品的价值

int dp[MAXV]; // dp[c]：容量为 c 时的最大价值


/*
 * 完全背包（一维滚动数组，容量正序循环）
 * 状态定义：dp[c] 表示背包容量为 c 时的最大价值。
 * 转移方程（容量正序更新）：
 *   dp[c] = max(dp[c], dp[c - v[i]] + w[i]),  c: v[i] -> V
 * 因为每种物品不限件数，正序更新时 dp[c - v[i]] 可能已经取过当前物品，
 * 从而允许同一物品被重复选取（完全背包特性）。
 * 与 0-1 背包唯一的区别即循环顺序（0-1 逆序，完全背包正序）。
 * 复杂度：O(n * V)，空间 O(V)。
 */
int knapComplete(int n, int V) {
    for (int i = 1; i <= n; ++i)
        for (int c = v[i]; c <= V; ++c)   // 容量正序

            dp[c] = max(dp[c], dp[c - v[i]] + w[i]);
    return dp[V];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：3 种物品（每种无限），背包容量 10，求最大价值

    int n = 3, V = 10;
    int vv[] = {2, 3, 4};
    int ww[] = {3, 4, 5};
    for (int i = 1; i <= n; ++i) { v[i] = vv[i - 1]; w[i] = ww[i - 1]; }

    cout << "完全背包最大价值：" << knapComplete(n, V) << '\n';
    // 输出每个容量下的最优价值

    for (int c = 1; c <= V; ++c)
        cout << "容量" << c << "=" << dp[c] << (c == V ? '\n' : ' ');
    return 0;
}
```

DP 状压DP

状压DP — 状态压缩 DP：TSP（类 P1433 吃奶酪）求最短回路

```cpp
// 状压DP — 状态压缩 DP：TSP（类 P1433 吃奶酪）求最短回路

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 16;        // 点数上限（n 较小时使用状压）

const double INF = 1e18;
double x[MAXN], y[MAXN];         // 各点坐标

double dis[MAXN][MAXN];          // dis[i][j]：点 i 到 j 的欧氏距离

double dp[1 << MAXN][MAXN];      // dp[mask][i]：已访问点集 mask，停在 i 的最短路径长


/*
 * 状态压缩 TSP（哈密顿回路：从点 0 出发遍历所有点并回到点 0）。
 * 状态：dp[mask][i] = 已访问的点集合为 mask（二进制位 1 表示已访问），
 *       且当前最后一次停靠在点 i 时的最短路径长度。
 * 转移：枚举下一个未访问的点 j：
 *   dp[mask | (1<<j)][j] = min(dp[mask|(1<<j)][j], dp[mask][i] + dis[i][j])
 * 初始：dp[1<<0][0] = 0（只访问过起点 0，停在 0）。
 * 答案：min_{i} dp[full][i] + dis[i][0]（遍历完所有点后回到起点）。
 * 复杂度：O(2^n * n^2)，仅适合 n <= 20 左右。
 */
double tsp(int n) {
    int full = (1 << n) - 1;
    for (int mask = 0; mask <= full; ++mask)
        for (int i = 0; i < n; ++i) dp[mask][i] = INF;
    dp[1][0] = 0;

    for (int mask = 1; mask <= full; ++mask) {
        for (int i = 0; i < n; ++i) {
            if (!(mask & (1 << i))) continue;    // i 必须已在集合中

            if (dp[mask][i] >= INF) continue;
            for (int j = 0; j < n; ++j) {
                if (mask & (1 << j)) continue;   // j 必须未访问

                int nmask = mask | (1 << j);
                dp[nmask][j] = min(dp[nmask][j], dp[mask][i] + dis[i][j]);
            }
        }
    }
    double ans = INF;
    for (int i = 1; i < n; ++i)                  // 最后停靠点 i，回到起点 0

        ans = min(ans, dp[full][i] + dis[i][0]);
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：4 个点（单位正方形顶点），求最短哈密顿回路长度

    int n = 4;
    double xs[] = {0, 0, 1, 1};
    double ys[] = {0, 1, 0, 1};
    for (int i = 0; i < n; ++i) { x[i] = xs[i]; y[i] = ys[i]; }
    for (int i = 0; i < n; ++i)
```

```
        for (int j = 0; j < n; ++j)
            dis[i][j] = hypot(x[i] - x[j], y[i] - y[j]);

    cout << "最短回路长度：" << tsp(n) << '\n'; // 正方形周长 4

    return 0;
}
```

DP

LCS

LCS — 最长公共子序列：O(n*m) 二维 DP

```cpp
// LCS — 最长公共子序列：O(n*m) 二维 DP

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 1005;
char s1[MAXN], s2[MAXN]; // 两个序列（下标从 1 开始）

int f[MAXN][MAXN];       // f[i][j]：s1[1..i] 与 s2[1..j] 的 LCS 长度


/*
 * 最长公共子序列 LCS（O(n * m)）。
 * 状态：f[i][j] 表示 s1 的前 i 个字符与 s2 的前 j 个字符的 LCS 长度。
 * 转移方程（按最后一个字符是否相等分类）：
 *   f[i][j] = f[i-1][j-1] + 1                  （s1[i] == s2[j]，匹配）
 *           = max(f[i-1][j], f[i][j-1])        （不匹配，取删除一方的最优）
 * 边界：f[0][*] = f[*][0] = 0。
 * 复杂度：O(n * m)，空间 O(n * m)。
 * 若只需长度可用滚动数组优化空间为 O(min(n,m))。
 */
int lcs(int n, int m) {
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j) {
            if (s1[i] == s2[j]) f[i][j] = f[i - 1][j - 1] + 1;
            else f[i][j] = max(f[i - 1][j], f[i][j - 1]);
        }
    return f[n][m];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：求两个字符串的 LCS 长度

    string a = "abcde", b = "ace";
    int n = (int)a.size(), m = (int)b.size();
    for (int i = 1; i <= n; ++i) s1[i] = a[i - 1];
    for (int i = 1; i <= m; ++i) s2[i] = b[i - 1];

    cout << "LCS 长度：" << lcs(n, m) << '\n'; // "ace" -> 3

    return 0;
}
```

DP LIS

LIS — 最长上升子序列：O(n logn) 用 tails + 二分；附 O(n^2) 版本注释

```cpp
// LIS — 最长上升子序列：O(n logn) 用 tails + 二分；附 O(n^2) 版本注释

#include <bits/stdc++.h>
using namespace std;

const int MAXN = 100005;
int a[MAXN]; // 输入序列


/*
 * 严格最长上升子序列（O(n log n)）。
 * tails[len] 维护"所有长度为 len 的上升子序列中结尾元素的最小值"。
 * 对每个 a[i]，在 tails 中二分查找第一个 >= a[i] 的位置 pos：
 *   - 若 pos == tails.size()（a[i] 大于所有结尾），说明能延长最长上升子序列，
 *     将 a[i] 追加到尾部；
 *   - 否则用 a[i] 替换 tails[pos]（更小的结尾元素永远更优，不改变长度）。
 * 最终 tails 的长度就是 LIS 长度。
 * 复杂度：O(n log n)，空间 O(n)。
 */
int lis(int n) {
    vector<int> tails;
    for (int i = 1; i <= n; ++i) {
        auto it = lower_bound(tails.begin(), tails.end(), a[i]);
        if (it == tails.end()) tails.push_back(a[i]);
        else *it = a[i];
    }
    return (int)tails.size();
}

/*
 * O(n^2) 版本（供参考）：
 * 状态：f[i] 表示以 a[i] 结尾的最长上升子序列长度。
 * 转移：f[i] = max(1, max_{j < i 且 a[j] < a[i]} f[j] + 1)。
 * 答案：max(f[i])。复杂度 O(n^2)。
 */

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：求序列的严格 LIS 长度

    int n = 8;
    int arr[] = {10, 9, 2, 5, 3, 7, 101, 18};
    for (int i = 1; i <= n; ++i) a[i] = arr[i - 1];

    cout << "LIS 长度：" << lis(n) << '\n'; // 如 2,3,7,101 或 2,3,7,18 -> 4

    return 0;
}
```

杂项

杂项

二分

二分 — 有序数组二分查找 + 二分答案（对单调谓词 check(mid) 判定）

```cpp
// 二分 — 有序数组二分查找 + 二分答案（对单调谓词 check(mid) 判定）

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// ---- 1. 二分查找：在有序数组 a[0..n-1] 中找 >= x 的第一个位置（等价于 std::lower_bound） ----

// 复杂度：O(log n)

int lower_bound_custom(const vector<int>& a, int x) {
    int l = 0, r = (int)a.size();   // 候选答案区间 [l, r)

    while (l < r) {
        int mid = l + (r - l) / 2;  // 避免 (l+r) 溢出

        if (a[mid] >= x) r = mid;   // mid 有可能就是答案，收缩右界

        else             l = mid + 1; // mid 太小，答案必在其右侧

    }
    return l;   // 返回第一个不小于 x 的下标；若都小于 x，则返回 a.size()

}

// ---- 2. 二分答案：对单调谓词 check(mid) 二分，求最小可行 x ----

// 判定函数能否把 b 分成不超过 k 段，且每段元素和 <= x

// 单调性：x 越大越容易成立（check 随 x 单调不减，是「假的→真的」）

// 复杂度：O(n log 值域)

bool check(const vector<int>& b, ll x, int k) {
    ll  cur = 0;   // 当前段累计和

    int cnt = 1;   // 已用段数（至少 1 段）

    for (int v : b) {
        if (v > x) return false;          // 单个元素已超限，必然不可行

        if (cur + v > x) { ++cnt; cur = v; } // 超出 x，开新段

        else               cur += v;
    }
    return cnt <= k;   // 能否用不超过 k 段装下全部元素

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示 1：二分查找第一个 >= x 的下标

    {
        vector<int> a = {1, 3, 3, 5, 7, 9};
        int x = 4;
        int pos = lower_bound_custom(a, x);
        cout << "第一个 >= " << x << " 的下标 = " << pos << ", 其值 = "
             << (pos < (int)a.size() ? a[pos] : -1) << "\n";
```

```
        // 下标 3，a[3] = 5

    }

    // 演示 2：二分答案——「最小化最大值」

    // 把数组分成不超过 k 段，求「各段和最大值」的最小可能值

    {
        vector<int> b = {2, 4, 1, 7, 3, 5};
        int k = 3;                  // 最多 3 段

        ll lo = 0, hi = 0;          // 答案一定在 [lo, hi] 内

        for (int v : b) {
            lo = max(lo, (ll)v);    // 至少是最大元素

            hi += v;                // 至多是全部之和（1 段）

        }
        // 找最小的可行 x。check 随 x 单调不减 → 用「向上取整的 mid」

        while (lo < hi) {
            ll mid = (lo + hi) / 2;
            if (check(b, mid, k)) hi = mid;   // mid 可行，尝试更小的 x

            else                  lo = mid + 1; // mid 不可行，必须更大

        }
        cout << "分成不超过 " << k << " 段时，最大段和的最小值 = " << lo << "\n";
    }

    return 0;
}
```

杂项

二维前缀和

二维前缀和 — 二维前缀和 O(1) 求任意子矩阵和

```cpp
// 二维前缀和 — 二维前缀和 O(1) 求任意子矩阵和

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 1000 + 5;   // 单个维度最大容量


int  n, m;          // 行数、列数（均从 1 开始编号）

int  a[MAXN][MAXN]; // 原矩阵

ll   s[MAXN][MAXN]; // 二维前缀和：s[i][j] = 左上角 (1,1) 到 (i,j) 子矩阵的元素和


// 预处理二维前缀和，O(n*m)

// 容斥原理：s[i][j] = s[i-1][j] + s[i][j-1] - s[i-1][j-1] + a[i][j]

void build() {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            s[i][j] = s[i - 1][j] + s[i][j - 1] - s[i - 1][j - 1] + a[i][j];
}

// O(1) 求子矩阵 (x1, y1)-(x2, y2) 的和（含边界）

// 也是容斥：总面积减去左右两块，再加上被重复减去的角落

ll sub_sum(int x1, int y1, int x2, int y2) {
    return s[x2][y2] - s[x1 - 1][y2] - s[x2][y1 - 1] + s[x1 - 1][y1 - 1];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：3 行 4 列的矩阵，求子矩阵 (2,2)-(3,4) 的和

    n = 3;
    m = 4;
    int val[4][5] = {
        {0, 0,  0,  0,  0},
        {0, 1,  2,  3,  4},
        {0, 5,  6,  7,  8},
        {0, 9, 10, 11, 12}
    };
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            a[i][j] = val[i][j];

    build();
    cout << "子矩阵 (2,2)-(3,4) 的和 = " << sub_sum(2, 2, 3, 4) << "\n";
    // 手算：(6+7+8) + (10+11+12) = 21 + 33 = 54


    return 0;
}
```

杂项 前缀和与差分

前缀和与差分 — 一维前缀和 O(1) 区间和 + 差分数组 O(1) 区间加、O(n) 单点查询

```cpp
// 前缀和与差分 — 一维前缀和 O(1) 区间和 + 差分数组 O(1) 区间加、O(n) 单点查询

#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MAXN = 100000 + 5;   // 数组最大容量


int  n;          // 元素个数（下标从 1 开始）

ll   a[MAXN];    // 原数组

ll   pre[MAXN];  // 前缀和：pre[i] = a[1] + a[2] + ... + a[i]

ll   diff[MAXN]; // 差分数组：diff[i] = a[i] - a[i-1]（约定 a[0] = 0）


// 构建前缀和，O(n)

void build_pre() {
    pre[0] = 0;
    for (int i = 1; i <= n; i++)
        pre[i] = pre[i - 1] + a[i];
}

// 用前缀和 O(1) 求闭区间 [l, r] 的元素和

ll range_sum(int l, int r) {
    return pre[r] - pre[l - 1];
}

// 构建差分数组：diff[i] = a[i] - a[i-1]（a[0] = 0），O(n)

void build_diff() {
    a[0] = 0;
    diff[0] = 0;
    for (int i = 1; i <= n; i++)
        diff[i] = a[i] - a[i - 1];
}

// 差分数组 O(1)：给闭区间 [l, r] 每个元素加上 val

void range_add(int l, int r, ll val) {
    diff[l] += val;
    if (r + 1 <= n) diff[r + 1] -= val;   // 在区间右侧之外抵消，保证还原时只影响 [l, r]

}

// 从差分数组还原出每个元素（等价于单点/整体查询），O(n)

// 原理：a[i] = a[i-1] + diff[i]，结果写回 a[]

void restore() {
    a[0] = 0;
    for (int i = 1; i <= n; i++)
        a[i] = a[i - 1] + diff[i];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);
```

```
    // 演示 1：前缀和求区间和

    n = 5;
    ll initArr[6] = {0, 1, 2, 3, 4, 5};   // a[1..5] = 1,2,3,4,5

    for (int i = 1; i <= n; i++) a[i] = initArr[i];
    build_pre();
    cout << "a[2..4] 的和 = " << range_sum(2, 4) << "\n";   // 2 + 3 + 4 = 9


    // 演示 2：差分数组做区间加

    build_diff();            // 由当前 a = {1,2,3,4,5} 建立差分

    range_add(3, 5, 10);     // 给 a[3..5] 各 +10

    restore();               // 从差分还原出每个元素的值

    cout << "区间 [3,5] 加 10 后的数组：";
    for (int i = 1; i <= n; i++) {
        cout << a[i];
        if (i < n) cout << ' ';
    }
    cout << "\n";

    return 0;
}
```

杂项

三分

三分 — 实数域三分法求单峰函数极值（迭代逼近极点）

```cpp
// 三分 — 实数域三分法求单峰函数极值（迭代逼近极点）

#include <bits/stdc++.h>
using namespace std;

// 单峰函数示例：f(x) = -(x-2)^2 + 5，在 [l, r] 上先增后减，顶点在 x = 2，最大值 5

// 实际使用时把 f 替换成具体的目标函数即可

double f(double x) {
    return -(x - 2) * (x - 2) + 5;
}

// 三分法在闭区间 [l, r] 上求单峰函数 f 的【最大值】所在位置

// 每次把区间缩短约 1/3，直到区间宽度小于 eps，收敛到极点

// 复杂度：O(log((r-l)/eps))，eps 为精度

// 说明：求最大值时比较 f(m1) 与 f(m2)，舍去函数值较? 一侧；求最小值则反之。

double ternary_search(double l, double r) {
    const double eps = 1e-9;
    while (r - l > eps) {
        double m1 = l + (r - l) / 3.0;   // 左三分点

        double m2 = r - (r - l) / 3.0;   // 右三分点

        if (f(m1) < f(m2))
            l = m1;   // 峰值在右侧三分段内，收缩左界

        else
            r = m2;   // 峰值在左侧三分段内（或 m1 处），收缩右界

    }
    return (l + r) / 2.0;   // 返回极值点 x

}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示：求 f(x) = -(x-2)^2 + 5 在 [-10, 10] 上的最大值

    double x = ternary_search(-10, 10);
    cout << fixed << setprecision(6);
    cout << "极值点 x ≈ " << x << "\n";          // 应接近 2

    cout << "最大值 f(x) ≈ " << f(x) << "\n";     // 应接近 5


    return 0;
}
```

杂项 **双指针**

双指针/滑动窗口 — 「元素和 <= k 的最长子数组」+ 有序数组两数之和

```cpp
// 双指针/滑动窗口 — 「元素和 <= k 的最长子数组」+ 有序数组两数之和

#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// 滑动窗口：正整数数组 a，求「元素和 <= k」的最长子数组长度

// 复杂度：O(n)，每个元素至多进窗口一次、出窗口一次

int longest_subarray_sum_le_k(const vector<int>& a, int k) {
    int  n = (int)a.size();
    int  l = 0, best = 0;
    ll   cur = 0;                       // 当前窗口 [l, r] 的元素和

    for (int r = 0; r < n; r++) {
        cur += a[r];                    // 右端点右移，把 a[r] 加入窗口

        while (cur > k) {               // 和超出 k，左端点右移收缩窗口

            cur -= a[l];
            ++l;
        }
        best = max(best, r - l + 1);    // 更新当前窗口长度

    }
    return best;
}

// 两数之和：在【有序】数组 a 中找和为 target 的两个下标（返回 {i, j}，无解返回 {-1,-1}）

// 复杂度：O(n)

pair<int, int> two_sum(const vector<int>& a, int target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        int s = a[l] + a[r];
        if (s == target) return {l, r};
        else if (s < target) ++l;   // 和太小，让左指针右移增大和

        else                 --r;   // 和太大，让右指针左移减小和

    }
    return {-1, -1};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);

    // 演示 1：滑动窗口 — 和 <= 8 的最长子数组

    {
        vector<int> a = {1, 3, 2, 5, 1, 1, 4, 2};
        int k = 8;
        cout << "和 <= 8 的最长子数组长度 = "
             << longest_subarray_sum_le_k(a, k) << "\n";
        // 最优区间为 [1,1,4,2]，长度 4

    }
```

```
    // 演示 2：两数之和（数组必须有序）

    {
        vector<int> b = {1, 3, 5, 7, 9};
        int target = 12;
        auto p = two_sum(b, target);
        cout << "两数之和为 12 的下标：" << p.first << " " << p.second << "\n";
        // 3 + 9 = 12 → 下标 1 和 4

    }

    return 0;
}
```

由 node gen-templates-html.js 生成 · 添加新模板后重跑即可更新本文档